The Machine Learning Field Guide

Every core model, the mathematics behind it,
the pictures that explain it, and when to use it

Supervised learning · Unsupervised learning · Ensembles & boosting
Metrics · Bias–variance · Regularisation · Cross-validation · Model selection

Prepared by Rahul Kothuri

How to read this guide

This guide is written to be read in two ways: front-to-back as a course, or as a reference you open before an interview or a project decision. Every model chapter follows the same template so you always know where to look:

1. **Intuition** – the idea in plain language (blue box).
2. **Mathematics** – the model equation, the loss it minimises, and how it is trained.
3. **Graphs** – decision boundaries, loss curves or geometry that make the maths visible.
4. **Model card** – task, the kind of data it fits, whether scaling is needed, key hyper-parameters, strengths and weaknesses.
5. **Best use cases** (green box) and **pitfalls** (red box).

Notation used throughout.

Symbol	Meaning
n, p	number of samples, number of features
$\mathbf{x}_i \in \mathbb{R}^p$, y_i	feature vector and target of sample i
$\mathbf{X} \in \mathbb{R}^{n \times p}$, $\mathbf{y} \in \mathbb{R}^n$	design matrix (rows = samples), target vector
$\mathbf{w}$, b (or $\boldsymbol{\beta}$, β_0)	weights / coefficients and intercept (bias)
$\hat{y}$, $\hat{p}$	prediction, predicted probability
$L(y, \hat{y})$, $J(\mathbf{w})$	per-sample loss, total cost (objective) function
λ (or α, $C = 1/\lambda$)	regularisation strength
η	learning rate
$\sigma(z) = 1/(1 + e^{-z})$	sigmoid function
$\mathbb{I}[\cdot]$	indicator: 1 if condition true, else 0
$\lVert\mathbf{w}\rVert_2^2 = \sum_j w_j^2$, $\lVert\mathbf{w}\rVert_1 = \sum_j \lvert w_j\rvert$	squared L2 norm, L1 norm

Code names refer to `scikit-learn` unless stated otherwise (e.g. `LogisticRegression(C=1.0)`). All charts in this guide were generated from real scikit-learn runs on synthetic or bundled datasets (moons, circles, blobs, Iris, Digits, Breast-Cancer, Diabetes), so the curves reflect genuine model behaviour.

Contents

How to read this guide **1**

1 Foundations of Machine Learning **4**
1.1 What machine learning is . . . 4
1.2 Types of learning . . . 4
1.3 Key dichotomies you will be asked about . . . 5
1.4 Types of data and what they imply . . . 6
1.5 The end-to-end ML workflow . . . 6

2 Data Preparation & Feature Engineering **7**
2.1 Splitting the data . . . 7
2.2 Missing values . . . 7
2.3 Feature scaling . . . 7
2.4 Encoding categorical variables . . . 8
2.5 Feature engineering . . . 8
2.6 Feature selection . . . 8
2.7 The curse of dimensionality . . . 9
2.8 Data leakage . . . 9

3 Loss Functions & Optimisation **10**
3.1 Loss functions at a glance . . . 10
3.2 Convexity . . . 10
3.3 Gradient descent . . . 11

4 Supervised Learning I: Regression Models **13**
4.1 Linear regression (Ordinary Least Squares) . . . 13
4.2 Polynomial regression . . . 14
4.3 Ridge regression (L2) . . . 15
4.4 Lasso regression (L1) . . . 15
4.5 Elastic Net (L1 + L2) . . . 15
4.6 Robust regression . . . 16
4.7 Support Vector Regression (SVR) . . . 16
4.8 Other regressors (covered in later chapters) . . . 16

5 Supervised Learning II: Classification Models **18**
5.1 Logistic regression . . . 18
5.2 K-Nearest Neighbours (KNN) . . . 20
5.3 Naive Bayes . . . 22
5.4 Linear & Quadratic Discriminant Analysis (LDA / QDA) . . . 23
5.5 Support Vector Machines (SVM) . . . 23
5.6 Perceptron and Multi-Layer Perceptron (neural network) . . . 26

6 Decision Trees & Ensemble Learning **28**
6.1 Decision trees (CART) . . . 28
6.2 Why ensembles work . . . 30
6.3 Bagging (Bootstrap Aggregating) . . . 30
6.4 Random Forest . . . 31
6.5 AdaBoost (Adaptive Boosting) . . . 32
6.6 Gradient Boosting Machines (GBM) . . . 32

6.7 XGBoost (Extreme Gradient Boosting) . . . 33
6.8 LightGBM and CatBoost . . . 35
6.9 Stacking, blending and voting . . . 36

7 Unsupervised Learning 37
7.1 K-Means clustering . . . 37
7.2 Hierarchical (agglomerative) clustering . . . 38
7.3 DBSCAN (density-based clustering) . . . 39
7.4 Gaussian Mixture Models (GMM) and EM . . . 39
7.5 Principal Component Analysis (PCA) . . . 41
7.6 t-SNE and UMAP (non-linear visualisation) . . . 42
7.7 Anomaly (outlier) detection . . . 43
7.8 Association rule mining (market-basket analysis) . . . 43

8 Evaluation Metrics 45
8.1 Regression metrics . . . 45
8.2 Classification metrics . . . 45
8.3 Clustering metrics . . . 48

9 Overfitting, Regularisation & Cross-Validation 49
9.1 Underfitting and overfitting . . . 49
9.2 The bias–variance decomposition . . . 49
9.3 Regularisation techniques . . . 51
9.4 Cross-validation . . . 52
9.5 Hyper-parameter tuning . . . 53
9.6 Imbalanced data . . . 54

10 Choosing the Right Model: Cheat Sheets 56
10.1 Same data, nine models . . . 56
10.2 Master model table . . . 56
10.3 Training and prediction cost . . . 57
10.4 A practical decision guide . . . 58
10.5 Hyper-parameter quick reference . . . 58

11 Interview Quick Revision 59

References & Further Reading 61

CHAPTER 1

Foundations of Machine Learning

1.1 What machine learning is

Machine learning (ML) builds a function f from data instead of writing the rules by hand. Tom Mitchell's classic definition: a program *learns* from experience E with respect to a task T and performance measure P if its performance on T, measured by P, improves with E.

Formally, we assume data are drawn i.i.d. (independent and identically distributed) from an unknown distribution $\mathcal{D}$ over $(\mathbf{x}, y)$. We want a model f with low **expected risk** (true error):

$$R(f) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}\big[L(y, f(\mathbf{x}))\big]. \tag{1.1}$$

Because $\mathcal{D}$ is unknown, we minimise the **empirical risk** on the training set, usually plus a complexity penalty Ω:

$$\hat{f} = \arg\min_{f\in\mathcal{F}} \underbrace{\frac{1}{n}\sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i))}_{\text{empirical risk (fit the data)}} + \underbrace{\lambda\,\Omega(f)}_{\text{regularisation (stay simple)}}. \tag{1.2}$$

Almost every algorithm in this guide is an instance of this one equation: it only changes the *model family* $\mathcal{F}$, the *loss* L, the *penalty* Ω and the *optimiser*. The gap between empirical risk and true risk is the **generalisation gap**; controlling it is what Chapter 9 is about.

1.2 Types of learning

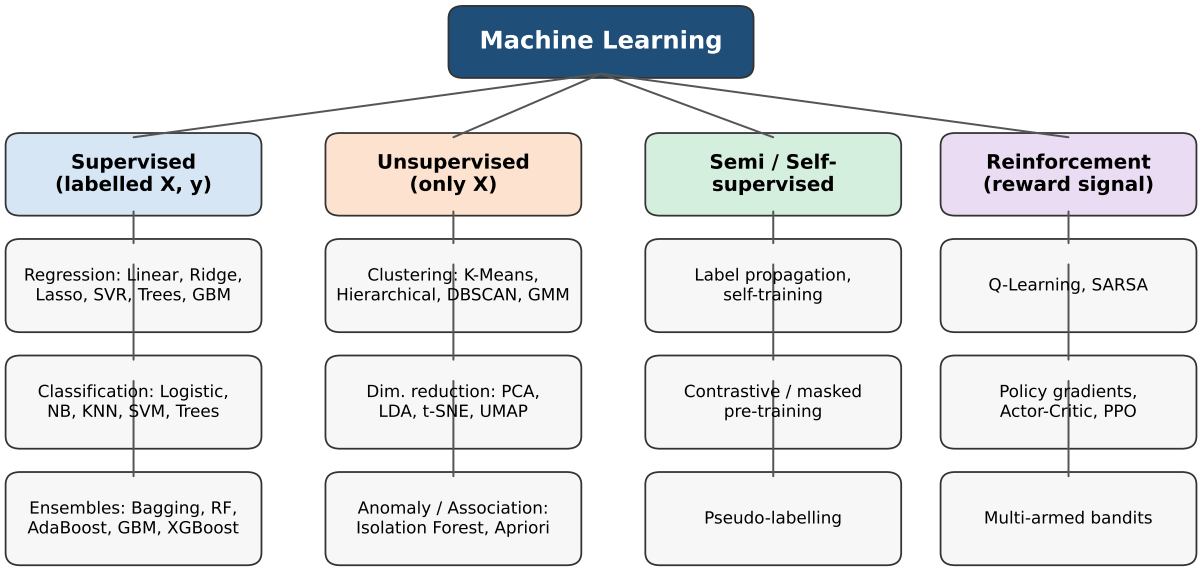


Figure 1.1: The machine learning landscape. This guide focuses on supervised and unsupervised learning.

Type	What you have	Typical goal / examples
Supervised	Inputs $\mathbf{x}$ *and* labels y	*Regression* ($y \in \mathbb{R}$: price, demand, load) and *classification* (y discrete: fraud/not, churn, disease class)
Unsupervised	Inputs $\mathbf{x}$ only	Find structure: clusters (customer segments), low-dimensional embeddings (PCA), anomalies, association rules
Semi-supervised	Few labels, many unlabelled	Use unlabelled data to improve a classifier (self-training, label propagation)
Self-supervised	Labels created from the data itself	Pre-train representations (masked-token prediction in LLMs, contrastive learning)
Reinforcement	Rewards from an environment	Learn a policy that maximises cumulative reward (games, robotics, recommendation)

1.3 Key dichotomies you will be asked about

Parametric vs non-parametric. Parametric models have a fixed number of parameters regardless of n (linear/logistic regression, Naive Bayes). Non-parametric models grow with the data (KNN stores all points; decision trees grow deeper; kernel SVMs keep support vectors). Parametric = strong assumptions, fast, less data needed; non-parametric = flexible, need more data, can overfit.

Discriminative vs generative. Discriminative models learn $P(y \mid \mathbf{x})$ or a boundary directly (logistic regression, SVM, trees). Generative models learn $P(\mathbf{x} \mid y)P(y)$ and use Bayes' rule (Naive Bayes, LDA/QDA, GMM). Generative models can generate data and handle missing features but rely on distributional assumptions.

Linear vs non-linear. A model is linear if its decision function is linear in the *parameters*. Polynomial regression is still a linear model (linear in $\boldsymbol{\beta}$) on non-linear features.

Batch vs online. Batch learning trains on the whole dataset; online learning updates incrementally (SGD, `partial_fit`).

Instance-based vs model-based. KNN memorises instances; most others compress data into parameters.

1.4 Types of data and what they imply

Data type	Description	Typical handling / good model families
Numerical continuous	Real values (income, temperature)	Scale for distance/gradient models; trees need no scaling
Numerical discrete / counts	Integers (visits, defects)	Poisson regression, tree ensembles; log1p transform
Categorical nominal	No order (city, product)	One-hot, target encoding, CatBoost native support
Categorical ordinal	Ordered (low/med/high)	Ordinal integer encoding
Binary	Yes/No	0/1 encoding; Bernoulli NB
Text	Sequences of tokens	TF-IDF + linear model / Multinomial NB; embeddings + transformers
Images, audio	High-dimensional signals	CNNs / pretrained deep models (beyond classical ML)
Time series	Ordered, autocorrelated	Lag/rolling features + GBM; time-aware CV (never shuffle)
Tabular mixed	Rows of mixed columns	Gradient-boosted trees are the default winner
Graph	Nodes/edges	Graph features or GNNs

1.5 The end-to-end ML workflow

1. **Frame the problem**: business objective → ML task (regression/classification/clustering), target y, unit of prediction, success metric and the *cost* of each error type.
2. **Collect and explore data** (EDA): distributions, missingness, outliers, class balance, correlations, leakage suspects.
3. **Split** into train / validation / test (or use cross-validation); the test set is touched once.
4. **Prepare features** inside a `Pipeline`: impute, encode, scale, engineer (Chapter 2).
5. **Baseline**: a dummy model (mean / majority class) and a simple model (linear/logistic).
6. **Model and tune**: try several families, tune with CV (Chapter 9).
7. **Evaluate** on held-out data with the right metric (Chapter 8); check per-segment errors.
8. **Deploy, monitor and retrain**: watch data drift ($P(\mathbf{x})$ changes), concept drift ($P(y \mid \mathbf{x})$ changes) and performance decay.

No Free Lunch theorem

Averaged over *all* possible problems, no learning algorithm is better than any other. In practice this means model choice must be driven by the structure of *your* data – which is why you always compare several families under the same cross-validation.

CHAPTER 2

Data Preparation & Feature Engineering

Models are only as good as the features they see. This chapter gives the formulas behind every common preprocessing step.

2.1 Splitting the data

- **Train** (fit parameters), **validation** (choose hyper-parameters / model), **test** (final unbiased estimate, used once). Common ratios: 60/20/20 or 70/15/15; with CV, a single train/test split such as 80/20.
- **Stratify** classification splits so class proportions are preserved.
- **Time series**: split by time (train on past, test on future), never randomly.
- **Grouped data** (several rows per patient/customer): split by group to avoid the same entity in both sets.

2.2 Missing values

- Mechanisms: MCAR (missing completely at random), MAR (depends on observed data), MNAR (depends on the missing value itself). Under MNAR, missingness is itself informative – add a binary *missing indicator*.
- Simple imputation: mean (sensitive to outliers), median (robust), mode/most-frequent (categorical), constant.
- Model-based: KNN imputation, iterative imputation (each feature regressed on the others, as in MICE).
- Tree boosters (XGBoost, LightGBM) learn a default direction for missing values natively.

2.3 Feature scaling

Distance-based (KNN, K-Means, SVM-RBF), gradient-based (linear/logistic regression, neural nets) and variance-based (PCA) methods are sensitive to feature scale; regularised models also need scaling so the penalty treats all coefficients fairly. Tree-based models are scale-invariant.

$$\text{Standardisation (z-score):} \quad x' = \frac{x - \mu}{\sigma} \qquad \text{mean 0, std 1} \tag{2.1}$$

$$\text{Min–max normalisation:} \quad x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \qquad \text{range } [0, 1] \tag{2.2}$$

$$\text{Robust scaling:} \quad x' = \frac{x - \operatorname{median}(x)}{Q_3 - Q_1} \qquad \text{resistant to outliers} \tag{2.3}$$

$$\text{Log / power transform:} \quad x' = \log(1 + x), \quad x' = \tfrac{x^{\lambda} - 1}{\lambda} \text{ (Box–Cox)} \qquad \text{reduce right skew} \tag{2.4}$$

$$\text{Unit-norm (row-wise):} \quad \mathbf{x}' = \mathbf{x} / \|\mathbf{x}\|_2 \qquad \text{text / cosine similarity} \tag{2.5}$$

Pitfalls & gotchas

Fit the scaler (its μ, σ) on the **training fold only** and apply it to validation/test. Fitting on all data leaks test statistics into training. Wrapping preprocessing and model in a `Pipeline` guarantees this, even inside cross-validation.

2.4 Encoding categorical variables

One-hot encoding creates K binary columns for K categories (or $K-1$ to avoid the "dummy-variable trap" – perfect multicollinearity – in unregularised linear models). Best for low-cardinality nominal features.

Ordinal encoding maps ordered categories to integers (low=0, medium=1, high=2). Fine for trees even on nominal data, but wrong for linear models on nominal data (implies a false order).

Target (mean) encoding replaces a category with a smoothed mean of the target:

$$\text{enc}(c) = \frac{n_c\,\bar{y}_c + m\,\bar{y}}{n_c + m}, \tag{2.6}$$

where n_c is the category count, $\bar{y}_c$ its target mean, $\bar{y}$ the global mean and m a smoothing weight. Must be computed out-of-fold to avoid leakage. Best for high-cardinality features (zip codes, product IDs).

Frequency / count encoding , **hashing trick** (fixed-size hashed columns), and **embeddings** (learned dense vectors) are alternatives at very high cardinality.

2.5 Feature engineering

- **Interactions and polynomials**: $x_1 x_2$, x_1^2 – let linear models capture curvature.
- **Binning**: turn a continuous variable into intervals (age groups) for non-linear effects in linear models.
- **Date/time**: hour, weekday, month, holiday flags; cyclical encoding $\left(\sin\frac{2\pi t}{T}, \cos\frac{2\pi t}{T}\right)$ so 23:00 is close to 00:00.
- **Aggregations**: per-customer means, counts, recency; **lags and rolling windows** for time series.
- **Text**: bag-of-words, TF-IDF $= \text{tf}(t,d)\cdot\left(\ln\frac{1+N}{1+\text{df}(t)}+1\right)$ (scikit-learn's smoothed form), n-grams, sentence embeddings.
- **Domain ratios**: debt/income, units per day, load factor – often the most valuable features.

2.6 Feature selection

Family	How it works	Examples
Filter	Score each feature independently of any model	Variance threshold, Pearson r, χ^2, ANOVA F, mutual information
Wrapper	Search feature subsets using model performance	Forward/backward selection, RFE (recursive feature elimination)
Embedded	Selection happens during training	Lasso (zero coefficients), tree importances, L1-logistic

Useful formulas:

$$\text{Pearson correlation:} \quad r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}} \in [-1, 1] \tag{2.7}$$

$$\text{Mutual information:} \quad I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)} \quad \text{(non-linear dependence)} \tag{2.8}$$

$$\text{Chi-square statistic:} \quad \chi^2 = \sum \frac{(O-E)^2}{E} \quad \text{(categorical feature vs class)} \tag{2.9}$$

$$\text{Variance inflation factor:} \quad \text{VIF}_j = \frac{1}{1 - R_j^2} \tag{2.10}$$

where R_j^2 comes from regressing x_j on all other features; VIF $>$ 5–10 signals problematic multi-collinearity.

2.7 The curse of dimensionality

As p grows, data become sparse and distances lose meaning. To capture a fraction r of uniformly distributed data in a p-dimensional unit hypercube, a sub-cube needs edge length

$$e_p(r) = r^{1/p}. \tag{2.11}$$

For $p = 10$, capturing just 10% of the data needs $e = 0.1^{1/10} \approx 0.80$ of the range of *every* feature – the neighbourhood is no longer "local". Consequences: KNN and K-Means degrade, more data are needed, overfitting risk rises. Remedies: feature selection, PCA, regularisation, domain knowledge.

2.8 Data leakage

Leakage is any information in training that would not be available at prediction time. Classic sources:

- **Target leakage**: a feature computed after (or from) the outcome, e.g. "refund_issued" when predicting fraud.
- **Train–test contamination**: scaling, imputing, SMOTE or target-encoding fitted on the full dataset before splitting.
- **Temporal leakage**: random splits on time series; rolling features that peek into the future.
- **Duplicate / group leakage**: the same entity in train and test.

Pitfalls & gotchas

A model with suspiciously high validation accuracy (e.g. 0.99 on a hard problem) almost always has leakage. Check feature importances: a single dominating feature is a red flag.

CHAPTER 3

Loss Functions & Optimisation

Training = choosing parameters $\boldsymbol{\theta}$ that minimise a cost $J(\boldsymbol{\theta})$. This chapter covers the losses and the optimisers used by almost every model.

3.1 Loss functions at a glance

Loss	Formula (per sample)	Used by / property
Squared error (L2)	$(y-\hat{y})^2$	Linear regression; predicts the conditional *mean*; outlier-sensitive
Absolute error (L1)	$\lvert y-\hat{y}\rvert$	Robust regression; predicts the conditional *median*
Huber	$\begin{cases}\frac{1}{2}r^2 & \lvert r\rvert \le \delta \\ \delta(\lvert r\rvert - \frac{1}{2}\delta) & \text{otherwise}\end{cases}$	Quadratic near 0, linear in tails – robust & smooth
Quantile (pinball)	$\max\big(\tau r, (\tau-1)r\big)$, $r = y-\hat{y}$	Prediction intervals; predicts the τ-quantile
Log loss / BCE	$-[y\log\hat{p} + (1-y)\log(1-\hat{p})]$	Logistic regression, boosting, neural nets
Categorical CE	$-\sum_k y_k \log \hat{p}_k$	Softmax / multiclass
Hinge	$\max(0,\, 1-yf(\mathbf{x}))$, $y\in\{-1,+1\}$	SVM; zero loss beyond the margin
Exponential	$e^{-yf(\mathbf{x})}$	AdaBoost

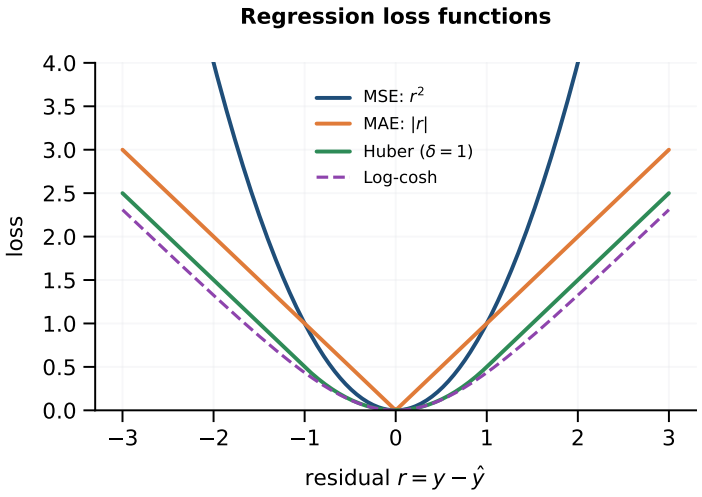


Figure 3.1: Regression losses. MSE grows quadratically, so a single outlier dominates; MAE and Huber grow linearly in the tails.

3.2 Convexity

A function is convex if $J(t\boldsymbol{a} + (1-t)\boldsymbol{b}) \le tJ(\boldsymbol{a}) + (1-t)J(\boldsymbol{b})$ for $t \in [0,1]$; equivalently its Hessian is positive semi-definite. For convex losses every local minimum is global. Linear, ridge, lasso, logistic regression and SVMs are convex; neural networks, K-Means and GMMs are not (they depend on initialisation).

3.3 Gradient descent

Move opposite to the gradient, the direction of steepest ascent:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \, \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t). \tag{3.1}$$

Variant	Gradient computed on	Trade-off
Batch GD	all n samples	Exact, smooth, slow per step on big data
Stochastic GD (SGD)	1 random sample	Very cheap, noisy; noise can escape shallow minima
Mini-batch GD	B samples (e.g. 32–512)	Standard in practice; vectorised and stable

3.3.1 Improved optimisers

$$\textbf{Momentum:} \quad \mathbf{v}_t = \beta \mathbf{v}_{t-1} + \eta \nabla J(\boldsymbol{\theta}_t), \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{v}_t \tag{3.2}$$

$$\textbf{RMSProp:} \quad \mathbf{s}_t = \rho \, \mathbf{s}_{t-1} + (1-\rho) \, \mathbf{g}_t^2, \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\mathbf{s}_t} + \epsilon} \, \mathbf{g}_t \tag{3.3}$$

$$\textbf{Adam:} \quad \mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}, \quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{3.4}$$

Typical Adam defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Momentum smooths the zig-zag across steep directions; RMSProp/Adam give each parameter its own adaptive step size.

3.3.2 Newton's method (second order)

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{H}^{-1} \nabla J(\boldsymbol{\theta}_t), \qquad \mathbf{H} = \nabla^2 J. \tag{3.5}$$

Uses curvature, converges in very few steps near the optimum, but each step costs $O(p^3)$. Logistic regression solvers (`newton-cg`, `lbfgs` as a quasi-Newton approximation) and XGBoost (per-leaf Newton step, Chapter 6) use this idea.

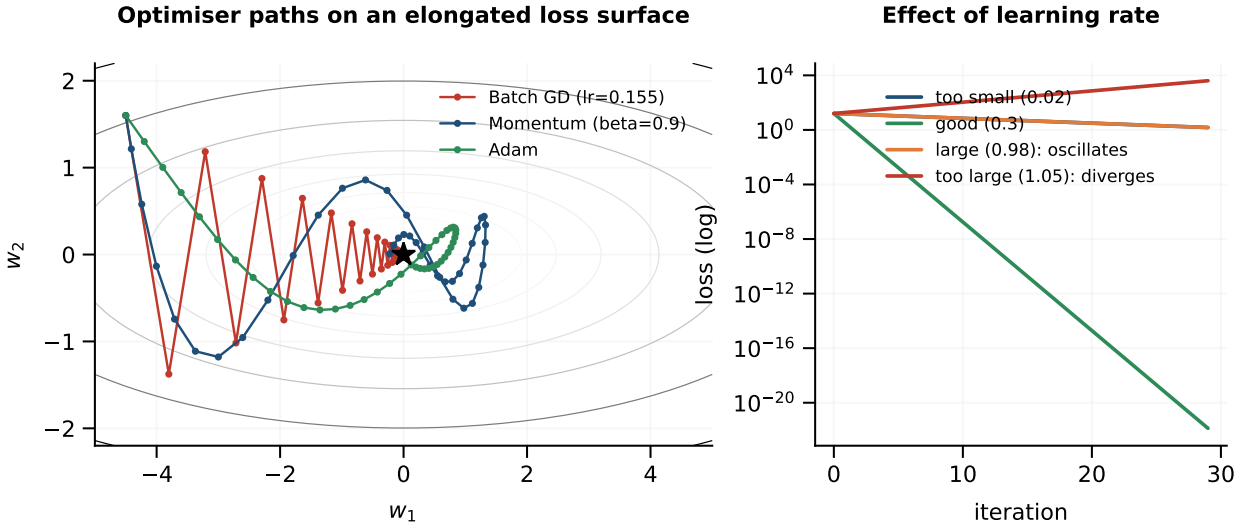


Figure 3.2: Left: on an elongated (ill-conditioned) loss surface plain GD zig-zags, momentum accelerates along the valley and Adam rescales each direction. Right: the learning rate decides whether training crawls, converges, oscillates or diverges.

Pitfalls & gotchas

- Unscaled features create elongated contours ⇒ slow, zig-zagging GD. Standardise first.
- Learning rate too high: loss explodes or oscillates; too low: training stalls. Use learning-rate schedules (step, cosine, warm-up) or adaptive optimisers.
- Monitor validation loss to stop early (Chapter 9).

CHAPTER 4

Supervised Learning I: Regression Models

Regression predicts a continuous target $y \in \mathbb{R}$.

4.1 Linear regression (Ordinary Least Squares)

Intuition

Fit the straight line (hyperplane) that makes the squared vertical distances between the points and the line as small as possible.

4.1.1 Model

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = \mathbf{x}^\top \boldsymbol{\beta} \qquad (\text{with } x_0 = 1). \tag{4.1}$$

Each β_j is the expected change in y for a one-unit increase in x_j, *holding the other features fixed.*

4.1.2 Cost function

$$J(\boldsymbol{\beta}) = \frac{1}{n}\sum_{i=1}^{n} \left(y_i - \mathbf{x}_i^\top \boldsymbol{\beta}\right)^2 = \frac{1}{n}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 \quad (\text{MSE}). \tag{4.2}$$

4.1.3 Closed-form solution (normal equation)

Set the gradient to zero:

$$\nabla_{\boldsymbol{\beta}} J = -\frac{2}{n}\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = 0 \implies \mathbf{X}^\top\mathbf{X}\boldsymbol{\beta} = \mathbf{X}^\top\mathbf{y} \implies \boxed{\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}} \tag{4.3}$$

Cost $O(np^2 + p^3)$; libraries use QR or SVD (pseudo-inverse) for numerical stability. If $\mathbf{X}^\top\mathbf{X}$ is singular (collinear features or $p > n$) the solution is not unique – ridge regression fixes this.

Simple regression ($p = 1$):

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = r\,\frac{s_y}{s_x}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}. \tag{4.4}$$

4.1.4 Gradient-descent solution

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \eta\,\frac{2}{n}\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}), \tag{4.5}$$

preferred when n or p is very large (`SGDRegressor`).

4.1.5 Probabilistic view

If $y = \mathbf{x}^\top\boldsymbol{\beta} + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, the log-likelihood is

$$\log \mathcal{L}(\boldsymbol{\beta}) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_i (y_i - \mathbf{x}_i^\top\boldsymbol{\beta})^2, \tag{4.6}$$

so **maximising the likelihood = minimising squared error**. This is why MSE is the "natural" loss under Gaussian noise.

4.1.6 Assumptions (LINE + no multicollinearity)

1. **L**inearity: $E[y \mid \mathbf{x}]$ is linear in the parameters (check residual-vs-fitted plot for curves).
2. **I**ndependence of errors (violated in time series – check Durbin–Watson).
3. **N**ormality of errors (needed for exact confidence intervals / p-values; check Q–Q plot).
4. **E**qual variance (homoscedasticity); a funnel-shaped residual plot signals heteroscedasticity.
5. No severe multicollinearity (VIF < 5–10); it inflates coefficient variance $\mathrm{Var}(\hat{\boldsymbol{\beta}}) = \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}$.

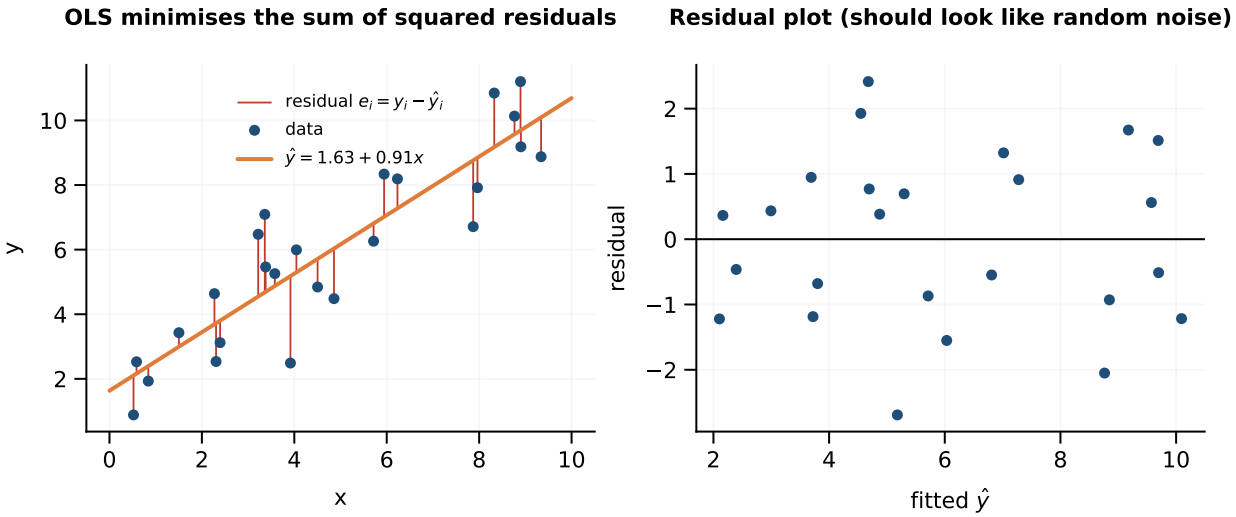


Figure 4.1: Left: OLS chooses the line that minimises the sum of squared residuals (red). Right: a healthy residual plot shows no pattern.

Task	Regression
Data it fits	Numerical features with a roughly linear relation to the target; works well for $n \gg p$; categorical features via one-hot
Needs scaling?	Not for OLS predictions, but yes for gradient descent and for regularised variants
Key hyper-parameters	None for OLS (`fit_intercept`)
Strengths	Fast, interpretable coefficients, closed-form, statistical inference (CIs, p-values), strong baseline
Weaknesses	Cannot capture non-linearity or interactions unless engineered; sensitive to outliers and multicollinearity

Best use cases & data fit

Price/demand forecasting baselines, effect estimation ("how much does y change per unit of x"), econometrics, any time interpretability matters more than raw accuracy.

4.2 Polynomial regression

Expand features, then run linear regression:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d. \tag{4.7}$$

Still linear in $\boldsymbol{\beta}$, so the normal equation applies. The degree d controls complexity: Figure 4.2 shows degree 1 underfitting, degree 4 fitting well and degree 15 overfitting. With p features, degree-d expansion creates $\binom{p+d}{d}$ columns – combine with regularisation.

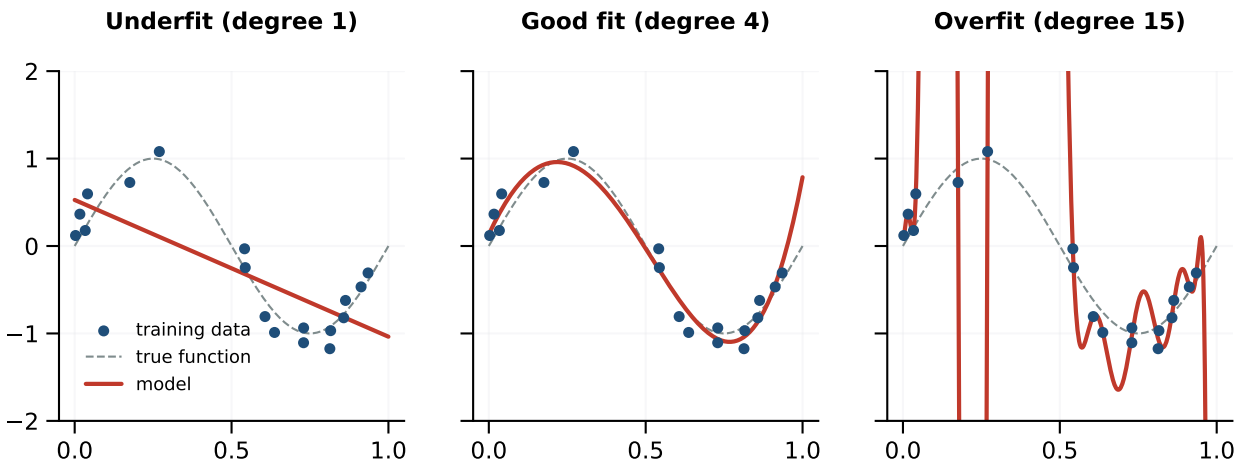


Figure 4.2: Polynomial regression on noisy samples of $\sin(2\pi x)$: the degree is the complexity knob.

4.3 Ridge regression (L2)

Intuition

Same as OLS, but every coefficient pays a "tax" proportional to its square, so the model prefers many small weights over a few huge ones.

$$J(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda\|\boldsymbol{\beta}\|_2^2 \qquad \Longrightarrow \qquad \boxed{\hat{\boldsymbol{\beta}}_{\text{ridge}} = (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}} \tag{4.8}$$

Adding $\lambda\mathbf{I}$ makes the matrix always invertible – ridge handles multicollinearity and $p > n$. Coefficients shrink towards (but never exactly to) zero. The intercept is not penalised. In scikit-learn, `Ridge(alpha=`λ`)`.

In terms of the SVD $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, ridge shrinks the component along the j-th principal direction by the factor $\frac{d_j^2}{d_j^2+\lambda}$: low-variance directions are shrunk the most.

4.4 Lasso regression (L1)

$$J(\boldsymbol{\beta}) = \frac{1}{2n}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \alpha\|\boldsymbol{\beta}\|_1 \qquad \text{(scikit-learn's form).} \tag{4.9}$$

No closed form (the absolute value is not differentiable at 0); solved by coordinate descent. To see why it produces zeros, take the textbook form $\frac{1}{2}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda\|\boldsymbol{\beta}\|_1$ with an orthonormal design ($\mathbf{X}^\top\mathbf{X} = \mathbf{I}$). The solution is **soft-thresholding** of the OLS estimate:

$$\hat{\beta}_j^{\text{lasso}} = \text{sign}(\hat{\beta}_j^{\text{OLS}})\,(|\hat{\beta}_j^{\text{OLS}}| - \lambda)_+, \tag{4.10}$$

so small coefficients become *exactly zero* $\Rightarrow$ automatic feature selection.

4.5 Elastic Net (L1 + L2)

$$J(\boldsymbol{\beta}) = \frac{1}{2n}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \alpha\rho\|\boldsymbol{\beta}\|_1 + \frac{\alpha(1-\rho)}{2}\|\boldsymbol{\beta}\|_2^2, \tag{4.11}$$

where $\rho =$`l1_ratio`. Combines sparsity with ridge's stability; with groups of correlated features it tends to keep the group together, whereas lasso picks one arbitrarily.

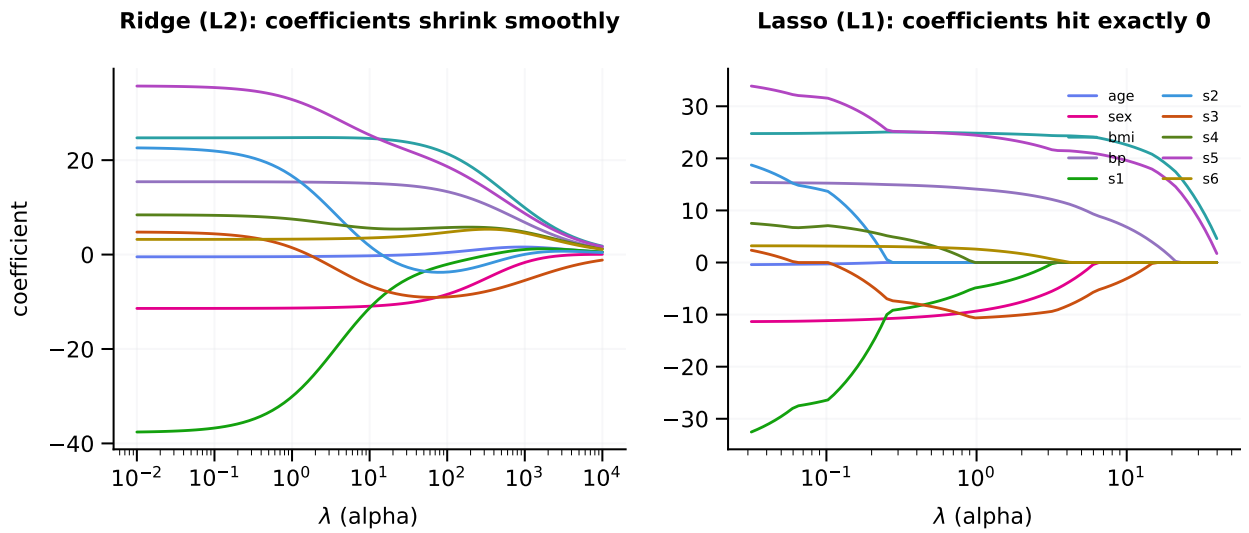


Figure 4.3: Regularisation paths on the Diabetes data (standardised). As λ grows, ridge coefficients shrink smoothly; lasso coefficients drop to exactly zero one by one.

	Ridge (L2)	Lasso (L1)	Elastic Net
Penalty	$\lambda \sum \beta_j^2$	$\lambda \sum \lvert\beta_j\rvert$	mix of both
Sparsity	No	Yes	Yes
Correlated features	Shares weight among them	Picks one, drops others	Keeps groups
Closed form	Yes	No	No
Use when	Many small effects, multicollinearity	Few relevant features, need selection	$p \gg n$ with correlated features

4.6 Robust regression

- **Huber regression** minimises the Huber loss (Chapter 3); outliers get linear rather than quadratic influence.
- **RANSAC** repeatedly fits on random subsets and keeps the model with the most inliers.
- **Quantile regression** models the τ-quantile – useful for prediction intervals and skewed targets.

4.7 Support Vector Regression (SVR)

SVR fits a function that stays within an ε-tube around the targets and is as flat as possible:

$$\min_{\mathbf{w},b,\xi,\xi^*} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i(\xi_i + \xi_i^*) \quad \text{s.t.} \quad \begin{cases} y_i - \mathbf{w}^\top\phi(\mathbf{x}_i) - b \le \varepsilon + \xi_i \\ \mathbf{w}^\top\phi(\mathbf{x}_i) + b - y_i \le \varepsilon + \xi_i^* \\ \xi_i, \xi_i^* \ge 0 \end{cases} \tag{4.12}$$

Equivalent to the **ε-insensitive loss** $\max(0, |y - f(\mathbf{x})| - \varepsilon)$: errors smaller than ε are ignored. Kernels (Section 5.5.4) make it non-linear. Good for small/medium, non-linear, low-noise data; scales poorly beyond $\sim 10^4$–10^5 samples; needs scaling.

4.8 Other regressors (covered in later chapters)

- **KNN regression**: $\hat{y} = \frac{1}{k}\sum_{i \in N_k(\mathbf{x})} y_i$ (Chapter 5).

- **Decision-tree regression**: piecewise-constant predictions, leaf value = mean of training targets in the leaf; splits chosen by variance (MSE) reduction (Chapter 6).
- **Random forest / gradient boosting regression**: the strongest default for tabular data (Chapter 6).
- **Generalised linear models (GLMs)**: $g(E[y]) = \mathbf{x}^\top \boldsymbol{\beta}$ with a link g. Poisson regression (log link) for counts, Gamma for positive skewed amounts, Tweedie for insurance claims.

CHAPTER 5

Supervised Learning II: Classification Models

Classification predicts a discrete label $y \in \{1, \dots, K\}$; most models also output class probabilities.

5.1 Logistic regression

Intuition

Compute a linear score, then squash it through the S-shaped sigmoid to get a probability between 0 and 1. The decision boundary $\mathbf{w}^\top \mathbf{x} + b = 0$ is a straight line (hyperplane).

5.1.1 Model

$$P(y = 1 \mid \mathbf{x}) = \hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = \mathbf{w}^\top \mathbf{x} + b. \tag{5.1}$$

Inverting gives the **log-odds (logit)**, which is linear in $\mathbf{x}$:

$$\log \frac{\hat{p}}{1 - \hat{p}} = \mathbf{w}^\top \mathbf{x} + b. \tag{5.2}$$

Interpretation: increasing x_j by one unit multiplies the odds by e^{w_j} (the *odds ratio*).

5.1.2 Loss: maximum likelihood ⇒ binary cross-entropy

Each label is Bernoulli: $P(y_i \mid \mathbf{x}_i) = \hat{p}_i^{\,y_i}(1 - \hat{p}_i)^{1 - y_i}$. Taking the negative log of the likelihood:

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \Big] \quad (+ \lambda \|\mathbf{w}\|_2^2). \tag{5.3}$$

This is convex. Using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, the gradient has the same elegant form as linear regression:

$$\nabla_{\mathbf{w}} J = \frac{1}{n} \mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y}), \qquad \mathbf{H} = \frac{1}{n} \mathbf{X}^\top \mathbf{S} \mathbf{X}, \quad \mathbf{S} = \operatorname{diag}\big(\hat{p}_i (1 - \hat{p}_i)\big). \tag{5.4}$$

There is no closed form; it is solved with gradient descent, Newton/IRLS or L-BFGS. **Why not MSE?** With a sigmoid, MSE is non-convex and gives tiny gradients for confidently wrong predictions.

5.1.3 Multiclass: softmax regression

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x} + b_k}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^\top \mathbf{x} + b_j}}, \qquad J = -\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}. \tag{5.5}$$

Alternative: one-vs-rest (OvR), training K binary classifiers.

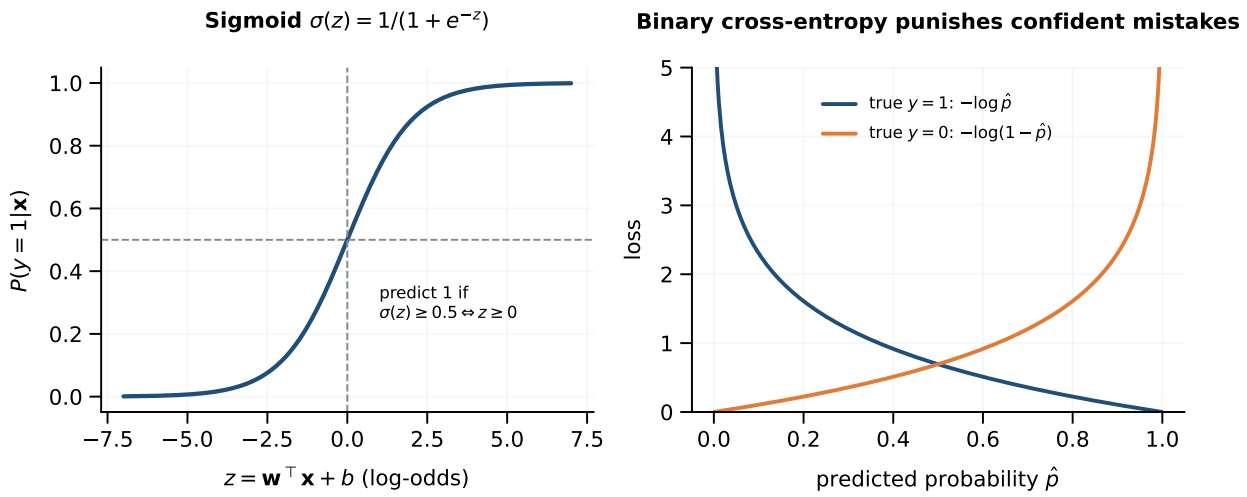


Figure 5.1: Left: the sigmoid maps log-odds to probability; the 0.5 threshold corresponds to $z = 0$. Right: cross-entropy grows without bound for confident wrong predictions.

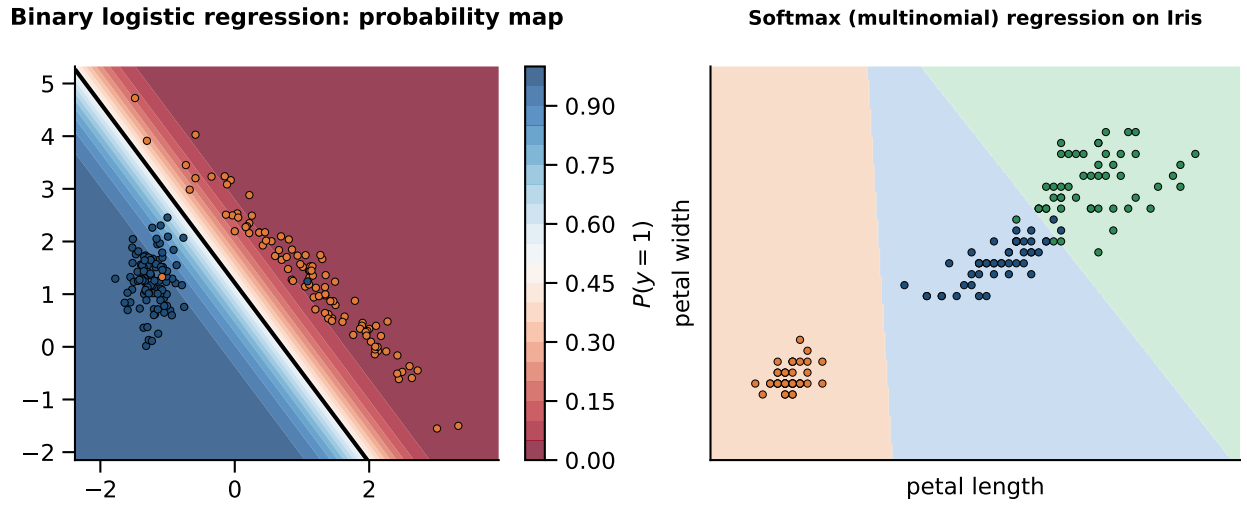


Figure 5.2: Left: logistic regression gives a probability surface with a linear 0.5 contour. Right: softmax regression yields piecewise-linear boundaries between three Iris classes.

Task	Binary & multiclass classification; probability estimation
Data it fits	Numerical + one-hot categorical, roughly linearly separable classes, high-dimensional sparse data (text TF-IDF)
Needs scaling?	Yes (for the solver and for fair regularisation)
Key hyper-parameters	`C` ($= 1/\lambda$), `penalty` (l1, l2, elasticnet), `class_weight`, `solver`
Strengths	Fast, well-calibrated probabilities, interpretable odds ratios, strong baseline, scales to millions of sparse features
Weaknesses	Linear boundary only (needs feature engineering for interactions); sensitive to multicollinearity and outliers

Best use cases & data fit

Credit scoring and churn (interpretability required), click-through prediction, spam/text classification on TF-IDF, medical risk scores, and as the baseline for every classification project.

5.2 K-Nearest Neighbours (KNN)

Intuition

"Tell me who your neighbours are." To classify a point, look at the k closest training points and take a vote (or average, for regression). Nothing is learned at training time – it is a *lazy*, instance-based learner.

5.2.1 Prediction rules

$$\text{Classification:} \quad \hat{y} = \arg\max_{c} \sum_{i \in N_k(\mathbf{x})} w_i \, \mathbb{I}[y_i = c], \tag{5.6}$$

$$\text{Regression:} \quad \hat{y} = \frac{\sum_{i \in N_k(\mathbf{x})} w_i y_i}{\sum_{i \in N_k(\mathbf{x})} w_i}, \tag{5.7}$$

with uniform weights $w_i = 1$ or distance weights $w_i = 1/d(\mathbf{x}, \mathbf{x}_i)$.

5.2.2 Distance metrics

$$\text{Minkowski: } d_p(\mathbf{x}, \mathbf{z}) = \Big(\sum_j |x_j - z_j|^p \Big)^{1/p} \qquad (p = 1 \text{ Manhattan}, \; p = 2 \text{ Euclidean}) \tag{5.8}$$

$$\text{Chebyshev } (p \to \infty)\text{: } d_\infty(\mathbf{x}, \mathbf{z}) = \max_j |x_j - z_j| \tag{5.9}$$

$$\text{Cosine distance: } 1 - \frac{\mathbf{x}^\top \mathbf{z}}{\|\mathbf{x}\| \|\mathbf{z}\|} \qquad \text{(text, embeddings)} \tag{5.10}$$

$$\text{Hamming: number of positions that differ (categorical / binary)} \tag{5.11}$$

Task	Classification and regression
Data it fits	Low-dimensional ($p \lesssim 20$) numerical data, small–medium n, irregular decision boundaries
Needs scaling?	**Yes, essential** – distances are dominated by large-scale features
Key hyper-parameters	`n_neighbors` (k), `weights`, `metric`, `p`
Strengths	No training, naturally multiclass, non-parametric, easy to explain
Weaknesses	Slow prediction $O(np)$ per query (use KD/Ball trees), memory-heavy, suffers from the curse of dimensionality, sensitive to irrelevant features and imbalance

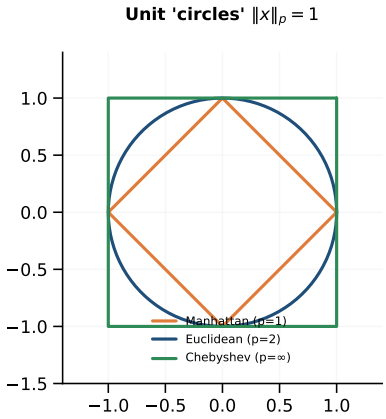


Figure 5.3: Unit balls of Minkowski distances.

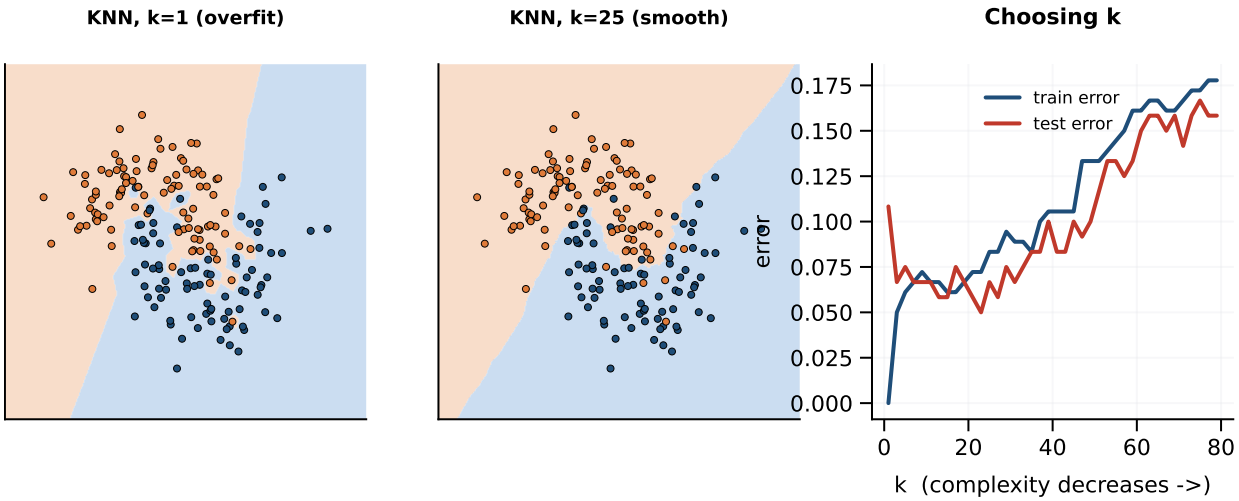


Figure 5.4: KNN on the moons data. $k = 1$ memorises noise (jagged boundary, zero training error); large k smooths. The test error is U-shaped in k – choose k by cross-validation (odd k avoids ties in binary problems).

Best use cases & data fit

Recommendation ("similar items"), small tabular problems, imputation (KNNImputer), anomaly detection by neighbour distance, quick baselines.

5.3 Naive Bayes

Intuition

Use Bayes' theorem and assume every feature is independent of the others *given the class*. The assumption is usually false, yet the classifier ranks classes surprisingly well – especially for text.

5.3.1 Bayes' theorem and the naive assumption

$$P(y = c \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y = c)\, P(y = c)}{P(\mathbf{x})} \quad \propto \quad P(y = c) \prod_{j=1}^{p} P(x_j \mid y = c), \tag{5.12}$$

$$\hat{y} = \arg\max_{c} \Big[\log P(y = c) + \sum_{j=1}^{p} \log P(x_j \mid y = c) \Big] \quad \text{(logs avoid underflow).} \tag{5.13}$$

5.3.2 Variants (choose by data type)

Variant	Likelihood $P(x_j \mid c)$	Data
Gaussian NB	$\frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\Big(- \frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2} \Big)$	Continuous features
Multinomial NB	$\hat{\theta}_{jc} = \frac{N_{jc} + \alpha}{N_c + \alpha p}$ (Laplace/additive smoothing)	Word counts, TF-IDF
Bernoulli NB	$p_{jc}^{x_j}(1 - p_{jc})^{1 - x_j}$	Binary presence/absence features
Complement NB	Estimates from all *other* classes	Imbalanced text
Categorical NB	Frequency table per category	Categorical features

Smoothing ($\alpha = 1$ is Laplace) prevents a single unseen word from making a probability exactly zero.

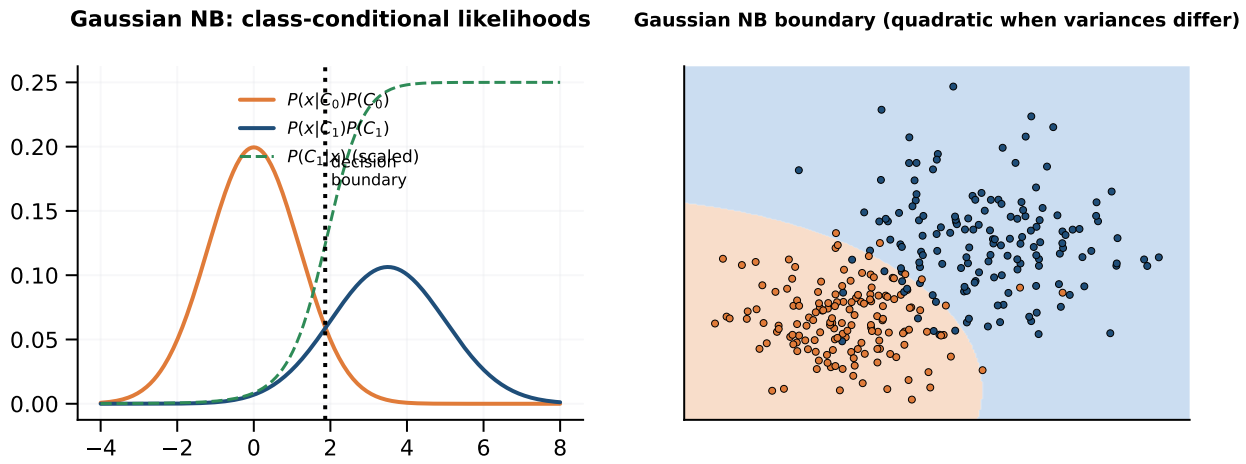


Figure 5.5: Left: Gaussian NB compares prior-weighted class likelihoods; the boundary is where they cross. Right: with different class variances the boundary becomes quadratic.

Task	Classification (often text)
Data it fits	High-dimensional sparse counts (text), categorical data, small training sets
Needs scaling?	No (Gaussian NB is per-feature; scaling has no effect)
Key hyper-parameters	`alpha` (smoothing), `var_smoothing`, class priors
Strengths	Extremely fast, works with little data, handles thousands of features, naturally multiclass, online learning
Weaknesses	Independence assumption ⇒ poorly calibrated (over-confident) probabilities; correlated features double-count evidence

Best use cases & data fit

Spam filtering, sentiment analysis, news/topic categorisation, ticket routing, real-time baselines.

5.4 Linear & Quadratic Discriminant Analysis (LDA / QDA)

Generative models assuming each class is Gaussian: $\mathbf{x} \mid y = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

- **LDA** assumes a shared covariance $\boldsymbol{\Sigma} \Rightarrow$ linear boundaries. Discriminant function:

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k - \tfrac{1}{2} \boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k + \log \pi_k, \qquad \hat{y} = \arg\max_k \delta_k(\mathbf{x}). \tag{5.14}$$

- **QDA** allows a different $\boldsymbol{\Sigma}_k$ per class $\Rightarrow$ quadratic boundaries:

$$\delta_k(\mathbf{x}) = -\tfrac{1}{2} \log |\boldsymbol{\Sigma}_k| - \tfrac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) + \log \pi_k. \tag{5.15}$$

- LDA is also a **supervised dimensionality reduction**: it projects onto at most $K - 1$ directions maximising Fisher's criterion $\dfrac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}$ (between-class over within-class scatter).

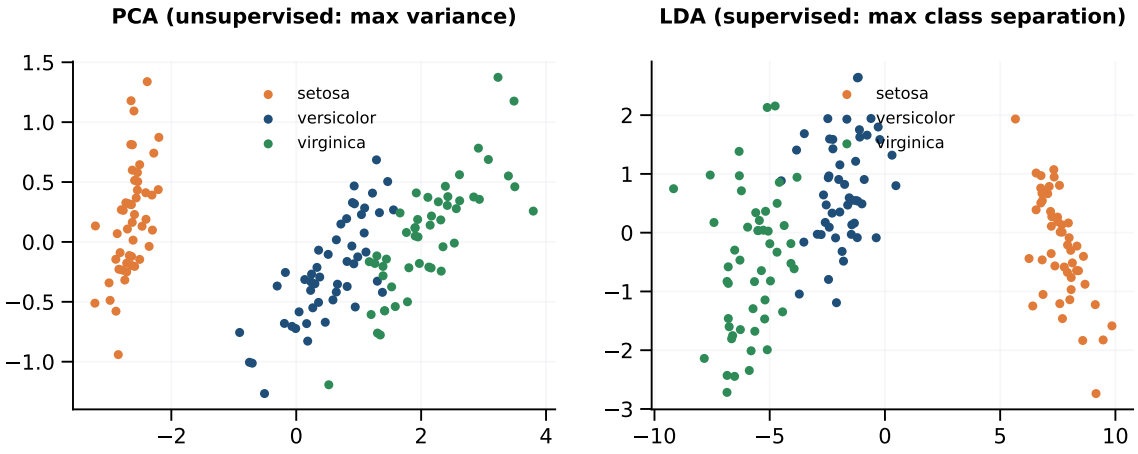


Figure 5.6: PCA ignores labels and keeps directions of maximum variance; LDA uses labels and finds the directions that separate classes best.

5.5 Support Vector Machines (SVM)

Intuition

Among all lines that separate the classes, choose the one with the *widest street* (margin) between them. Only the points on the edge of the street – the *support vectors* – determine the boundary.

5.5.1 Hard-margin SVM (separable data)

Labels $y_i \in \{-1, +1\}$, decision function $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$. The distance between the hyperplanes $\mathbf{w}^\top \mathbf{x} + b = \pm 1$ is $\dfrac{2}{\|\mathbf{w}\|}$, so maximising the margin means:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i. \tag{5.16}$$

5.5.2 Soft-margin SVM (real data)

Slack variables $\xi_i \geq 0$ allow violations, priced by C:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \tag{5.17}$$

Equivalently, unconstrained **hinge loss + L2 regularisation**:

$$\min_{\mathbf{w},b} \sum_{i=1}^{n} \max\left(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right) + \frac{1}{2C}\|\mathbf{w}\|^2. \tag{5.18}$$

Large $C \Rightarrow$ few violations, narrow margin, risk of overfitting; small $C \Rightarrow$ wide margin, more regularisation.

5.5.3 Dual problem and support vectors

Using Lagrange multipliers α_i:

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0, \tag{5.19}$$

$$f(\mathbf{x}) = \sum_{i \in SV} \alpha_i y_i \, \mathbf{x}_i^\top \mathbf{x} + b. \tag{5.20}$$

Only points with $\alpha_i > 0$ (the support vectors) matter. The data appear *only through dot products* – this enables the kernel trick.

5.5.4 The kernel trick

Replace $\mathbf{x}_i^\top \mathbf{x}_j$ with a kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$, an inner product in a (possibly infinite-dimensional) feature space, computed without ever building ϕ:

Kernel	$K(\mathbf{x}, \mathbf{z})$	Notes
Linear	$\mathbf{x}^\top \mathbf{z}$	High-dim sparse data (text)
Polynomial	$(\gamma\, \mathbf{x}^\top \mathbf{z} + r)^d$	Feature interactions of degree d
RBF (Gaussian)	$\exp(-\gamma\|\mathbf{x} - \mathbf{z}\|^2)$	Default non-linear kernel; infinite-dim ϕ
Sigmoid	$\tanh(\gamma\, \mathbf{x}^\top \mathbf{z} + r)$	Neural-net-like; rarely best

For RBF, γ sets the reach of each support vector: small γ = smooth, almost linear boundary; large γ = each point is an island (overfit). Tune C and γ jointly on a log grid.

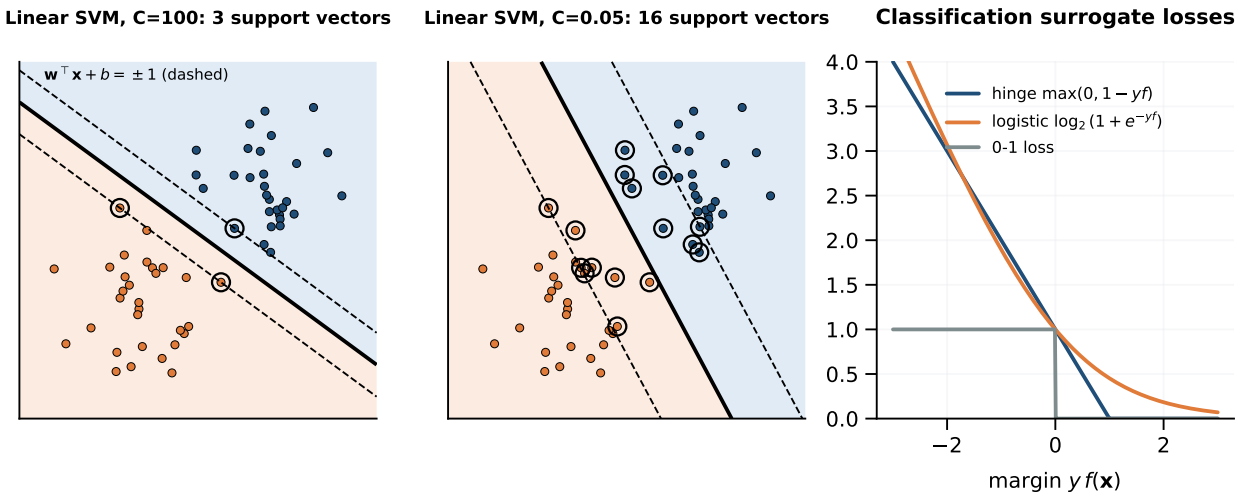


Figure 5.7: Left/middle: a large C gives a narrow margin with few support vectors (circled); a small C widens the margin and uses many. Right: the hinge loss is a convex upper bound on the 0-1 loss, and is exactly zero once the margin $yf \geq 1$.

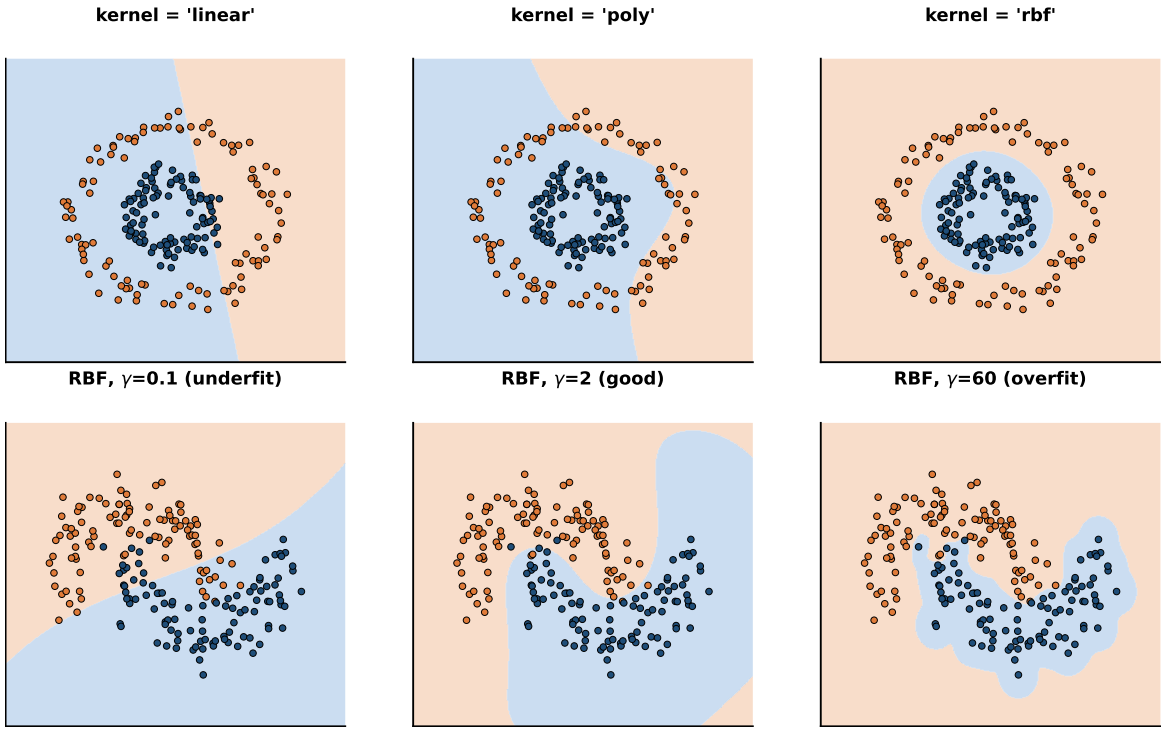


Figure 5.8: Top: on concentric circles the linear kernel fails while polynomial and RBF kernels succeed. Bottom: the RBF γ moves the model from underfitting to overfitting.

Task	Classification (SVC), regression (SVR), novelty detection (One-Class SVM)
Data it fits	Small–medium datasets ($n \lesssim 10^5$), high-dimensional data (text, genomics) with linear kernel, complex but smooth boundaries with RBF
Needs scaling?	**Yes, essential**
Key hyper-parameters	`C`, `kernel`, `gamma`, `degree`, `class_weight`
Strengths	Maximum-margin generalisation, effective when $p > n$, memory-efficient (stores SVs only), flexible kernels
Weaknesses	Kernel training is $O(n^2)$–$O(n^3)$; no native probabilities (Platt scaling needed); sensitive to C, γ; less interpretable

Best use cases & data fit

Text categorisation, image classification with hand-crafted features, bioinformatics, handwriting recognition, any small-but-complex dataset.

5.6 Perceptron and Multi-Layer Perceptron (neural network)

5.6.1 Perceptron

$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$; on each mistake update $\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i \mathbf{x}_i$. Converges only if the data are linearly separable (it cannot learn XOR).

5.6.2 Multi-layer perceptron (MLP)

Stacking layers with non-linear activations makes a universal function approximator:

$$\mathbf{a}^{(0)} = \mathbf{x}, \qquad \mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = g(\mathbf{z}^{(l)}), \qquad \hat{\mathbf{y}} = \text{softmax}\left(\mathbf{z}^{(L)}\right). \tag{5.21}$$

Backpropagation applies the chain rule from the output backwards:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} J \odot g'(\mathbf{z}^{(L)}), \qquad \boldsymbol{\delta}^{(l)} = \left(\mathbf{W}^{(l+1)\top}\boldsymbol{\delta}^{(l+1)}\right) \odot g'(\mathbf{z}^{(l)}), \qquad \frac{\partial J}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)}\mathbf{a}^{(l-1)\top}. \tag{5.22}$$

(For softmax + cross-entropy, $\boldsymbol{\delta}^{(L)}$ simplifies to $\hat{\mathbf{y}} - \mathbf{y}$.)

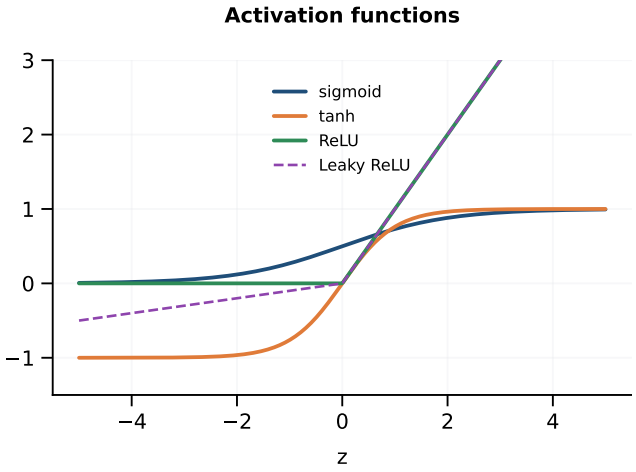


Figure 5.9: Common activation functions. ReLU, $\max(0, z)$, avoids the vanishing gradients of sigmoid/tanh in deep networks.

Task	Classification and regression
Data it fits	Large datasets with complex non-linear patterns; images, audio, text (deep variants: CNN, RNN, Transformer)
Needs scaling?	**Yes**
Key hyper-parameters	`hidden_layer_sizes`, `activation`, `alpha` (L2), `learning_rate_init`, `batch_size`, `early_stopping`
Strengths	Learns features automatically; most flexible model family
Weaknesses	Data- and compute-hungry, many hyper-parameters, non-convex, least interpretable; on small tabular data usually beaten by boosted trees

CHAPTER 6

Decision Trees & Ensemble Learning

6.1 Decision trees (CART)

Intuition

Play "20 questions" with the data: repeatedly ask the yes/no question ("is petal length $\leq$ 2.45?") that makes the resulting groups as pure as possible. Each leaf predicts the majority class (classification) or the mean (regression).

6.1.1 Impurity measures (classification)

For a node with class proportions $p_1, \ldots, p_K$:

$$\text{Gini impurity:} \quad G = 1 - \sum_{k=1}^{K} p_k^2 \qquad \text{(probability of mislabelling a random sample)} \tag{6.1}$$

$$\text{Entropy:} \quad H = -\sum_{k=1}^{K} p_k \log_2 p_k \qquad (0 = \text{pure};\ \log_2 K = \text{uniform}) \tag{6.2}$$

$$\text{Misclassification error:} \quad E = 1 - \max_k p_k \tag{6.3}$$

6.1.2 Choosing a split

For a split of parent P (n samples) into children L (n_L) and R (n_R), choose the feature and threshold that maximise the impurity decrease:

$$\Delta I = I(P) - \Big(\frac{n_L}{n} I(L) + \frac{n_R}{n} I(R)\Big). \tag{6.4}$$

With entropy, ΔI is the **information gain**. C4.5 uses the **gain ratio** IG/SplitInfo, with SplitInfo $= -\sum_j \frac{n_j}{n} \log_2 \frac{n_j}{n}$, to avoid favouring many-valued features.

Regression trees use variance (MSE) reduction:

$$\Delta = \mathrm{MSE}(P) - \Big(\frac{n_L}{n}\mathrm{MSE}(L) + \frac{n_R}{n}\mathrm{MSE}(R)\Big), \qquad \mathrm{MSE}(S) = \frac{1}{|S|}\sum_{i \in S}(y_i - \bar{y}_S)^2. \tag{6.5}$$

Worked example (entropy). A parent node has 10 samples: 5 positive, 5 negative, so $H = 1$ bit. A split gives $L = \{4+, 1-\}$ and $R = \{1+, 4-\}$. Each child has $H = -0.8 \log_2 0.8 - 0.2 \log_2 0.2 = 0.722$. Information gain $= 1 - (0.5 \cdot 0.722 + 0.5 \cdot 0.722) = 0.278$ bits. Gini: parent 0.5, children $1 - 0.8^2 - 0.2^2 = 0.32$, decrease $= 0.18$.

6.1.3 Controlling complexity

An unrestricted tree grows until leaves are pure $\Rightarrow$ zero training error and high variance (Figure 6.3).

- **Pre-pruning**: `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`, `min_impurity_decrease`.
- **Post-pruning (cost-complexity)**: grow a full tree, then minimise

$$R_\alpha(T) = R(T) + \alpha |T|, \tag{6.6}$$

  where $R(T)$ is the training error and $|T|$ the number of leaves; choose α (`ccp_alpha`) by CV.

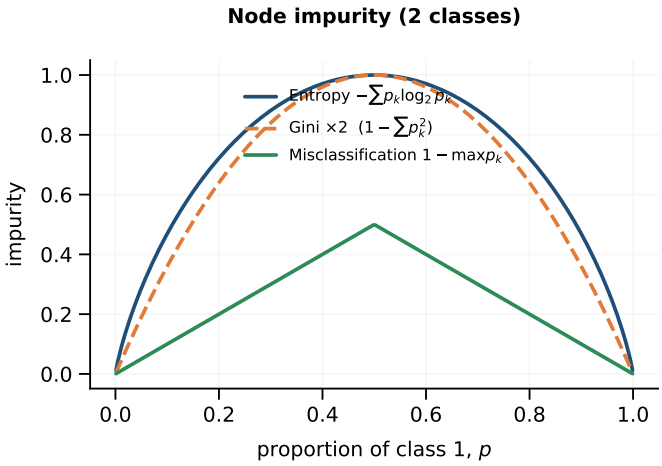


Figure 6.1: Impurity measures for two classes are all maximal at $p = 0.5$ and zero for pure nodes; Gini and entropy are smooth and almost proportional.

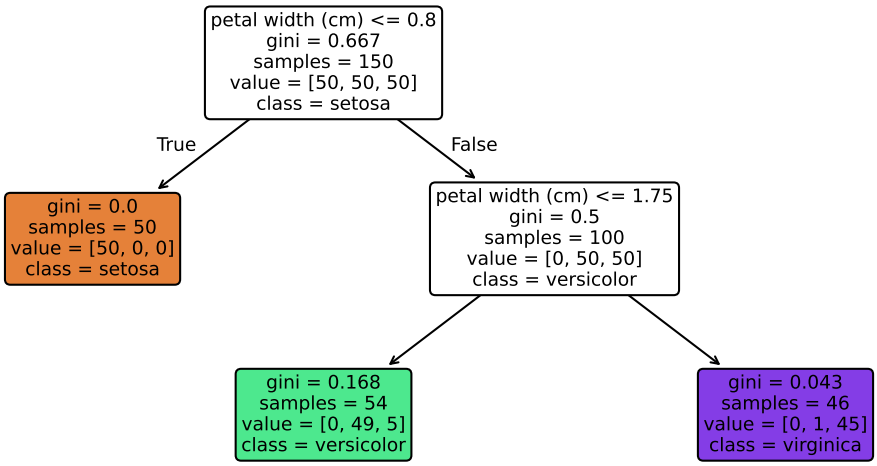


Figure 6.2: A depth-2 tree on Iris. Each node shows the split rule, Gini, number of samples and class counts.

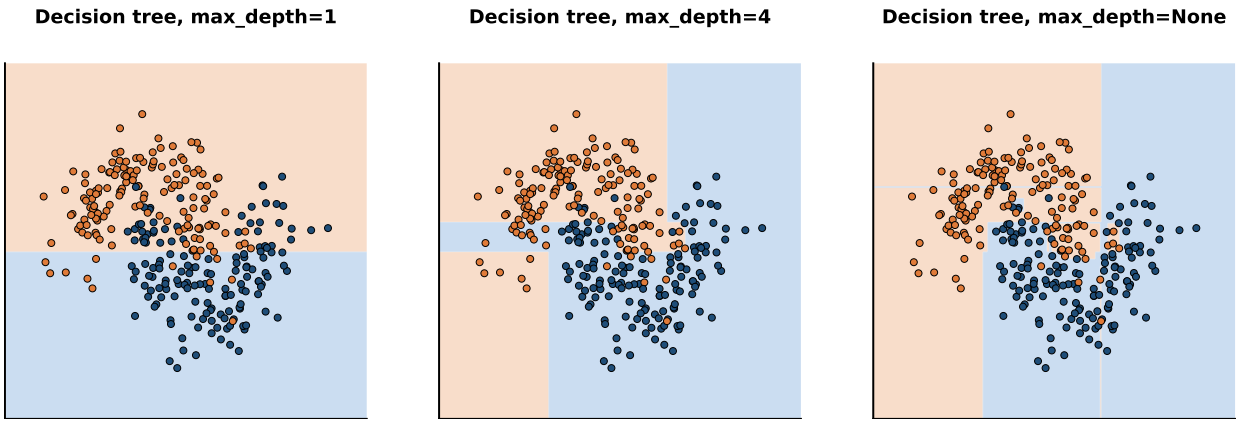


Figure 6.3: Depth controls the bias–variance trade-off: a stump underfits, depth 4 is reasonable, an unlimited tree carves out islands around single noisy points.

Task	Classification and regression
Data it fits	Tabular data with mixed numerical/categorical features, non-linear relations, interactions, thresholds
Needs scaling?	**No** – splits are scale-invariant
Key hyper-parameters	`max_depth`, `min_samples_leaf`, `criterion`, `ccp_alpha`
Strengths	Interpretable rules, captures interactions automatically, little preprocessing, fast prediction
Weaknesses	High variance (small data changes ⇒ different tree), axis-aligned staircase boundaries, greedy (not globally optimal), poor extrapolation

Best use cases & data fit

Explainable rule-based decisions (loan approval, triage), quick insight into feature thresholds, and above all as the base learner for random forests and boosting.

6.2 Why ensembles work

Average B models, each with variance σ^2 and pairwise correlation ρ:

$$\operatorname{Var}\left(\frac{1}{B}\sum_{b=1}^{B} f_b(\mathbf{x})\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2. \tag{6.7}$$

More models shrink the second term; *de-correlating* the models shrinks the first. Bagging and random forests attack **variance**; boosting attacks **bias** by adding models sequentially that fix the current errors.

	Bagging / Random Forest	Boosting
Training	Parallel, independent models	Sequential, each fixes predecessors' errors
Data per model	Bootstrap sample	Re-weighted samples (AdaBoost) or residuals/gradients (GBM)
Base learner	Deep, low-bias / high-variance trees	Shallow, high-bias trees (stumps, depth 3–8)
Reduces	Variance	Bias (and variance)
Overfitting risk	Low; more trees never hurt	Can overfit with too many rounds – use shrinkage + early stopping
Noisy labels	Robust	Sensitive (AdaBoost especially)
Examples	Bagging, Random Forest, Extra Trees	AdaBoost, GBM, XGBoost, LightGBM, CatBoost

6.3 Bagging (Bootstrap Aggregating)

Algorithm: Bagging

For $b = 1, \ldots, B$: draw a bootstrap sample (size n, *with replacement*) and fit a model f_b. Predict by majority vote (classification) or average (regression): $\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B}\sum_b f_b(\mathbf{x})$.

The probability a given sample is *not* in a bootstrap sample is

$$\left(1 - \frac{1}{n}\right)^n \xrightarrow{n\to\infty} e^{-1} \approx 0.368, \tag{6.8}$$

so each model sees about 63.2% of unique samples. The left-out 36.8% are **out-of-bag (OOB)** samples: predicting each point with only the trees that did not see it gives a free validation estimate (`oob_score=True`).

6.4 Random Forest

Intuition

Bagging of trees plus one extra twist: at every split, a tree may only consider a *random subset* of m features. This stops all trees from using the same dominant feature at the top, de-correlates them (lower ρ) and lowers the ensemble variance.

- Typical m: $\sqrt{p}$ for classification, $p/3$ for regression (Breiman's recommendations; scikit-learn's regressor default is all features, `max_features=1.0`). With $m = p$, a random forest equals bagged trees.
- Trees are grown deep (low bias); averaging removes variance.
- **Feature importance**: mean decrease in impurity (MDI, fast but biased towards high-cardinality features) or **permutation importance** (drop in score when a feature is shuffled – more reliable).
- **Extra Trees** (extremely randomised trees) also draw split *thresholds* at random: even lower variance, faster training.

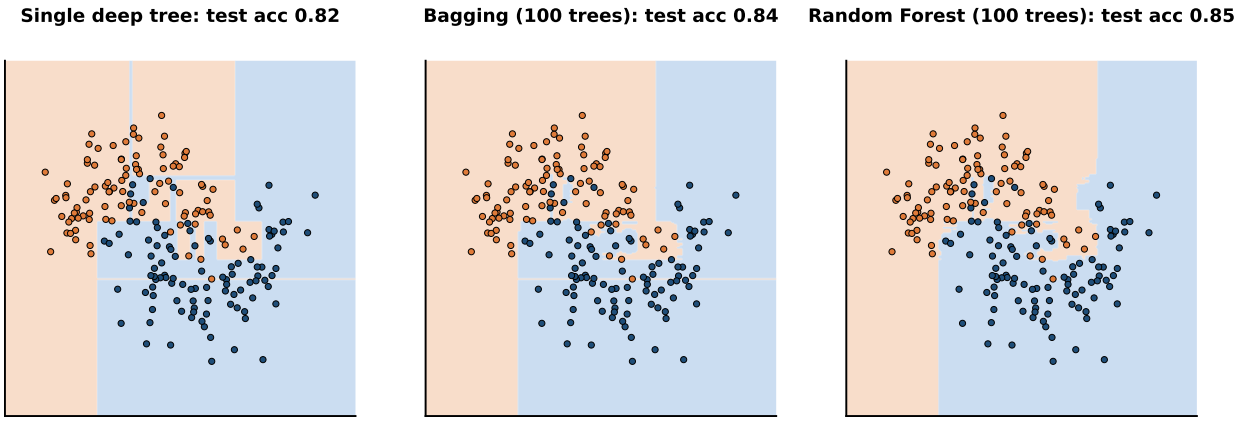


Figure 6.4: A single deep tree overfits; bagging averages 100 trees into a smoother boundary; a random forest de-correlates the trees further.

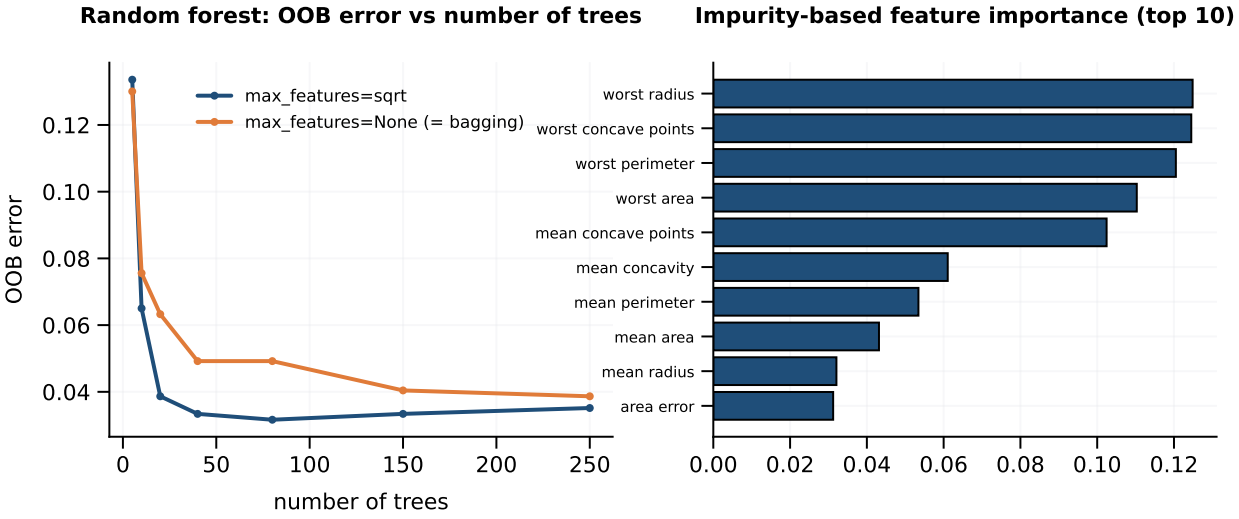


Figure 6.5: Left: OOB error falls and plateaus as trees are added – more trees never cause overfitting, they only cost time. Right: impurity-based importances on the Breast-Cancer data.

Task	Classification and regression
Data it fits	Tabular data of almost any kind; robust to outliers, non-linearities, interactions, irrelevant features
Needs scaling?	No
Key hyper-parameters	`n_estimators`, `max_features`, `max_depth`, `min_samples_leaf`, `class_weight`
Strengths	Strong out-of-the-box accuracy, hard to overfit, OOB estimate, parallel, feature importance, little tuning
Weaknesses	Larger and slower than one tree, less interpretable, cannot extrapolate beyond the training range, usually slightly below tuned boosting

Best use cases & data fit

Default first "serious" model for tabular data: fraud and risk scoring, churn, medical diagnosis, feature-importance analysis, bioinformatics.

6.5 AdaBoost (Adaptive Boosting)

Intuition

Train a weak learner, then *increase the weight of the samples it got wrong* so the next learner focuses on the hard cases. The final prediction is a weighted vote in which accurate learners count more.

Algorithm: AdaBoost (binary, $y_i \in \{-1, +1\}$)

1. Initialise weights $w_i = 1/n$.
2. For $m = 1, \ldots, M$:
   (a) Fit weak learner $G_m(\mathbf{x})$ (e.g. a stump) using weights w_i.
   (b) Weighted error: $\varepsilon_m = \dfrac{\sum_i w_i \, \mathbb{I}[y_i \neq G_m(\mathbf{x}_i)]}{\sum_i w_i}$.
   (c) Learner weight: $\alpha_m = \dfrac{1}{2} \ln \dfrac{1 - \varepsilon_m}{\varepsilon_m}$.
   (d) Update: $w_i \leftarrow w_i \exp\big(-\alpha_m \, y_i \, G_m(\mathbf{x}_i)\big)$, then normalise so $\sum_i w_i = 1$. (Misclassified points are multiplied by e^{α_m}, correct ones by $e^{-\alpha_m}$.)
3. Output $F(\mathbf{x}) = \operatorname{sign}\Big(\sum_{m=1}^{M} \alpha_m G_m(\mathbf{x})\Big)$.

AdaBoost is equivalent to forward stagewise additive modelling with the **exponential loss** $L = e^{-yF(\mathbf{x})}$, which explains its sensitivity to outliers and label noise. The multiclass version (SAMME) uses $\alpha_m = \ln \frac{1-\varepsilon_m}{\varepsilon_m} + \ln(K-1)$.

6.6 Gradient Boosting Machines (GBM)

Intuition

Each new tree is fitted to the *residual errors* of the current ensemble. More generally, it fits the *negative gradient* of the loss – gradient descent in function space, where each step is a tree.

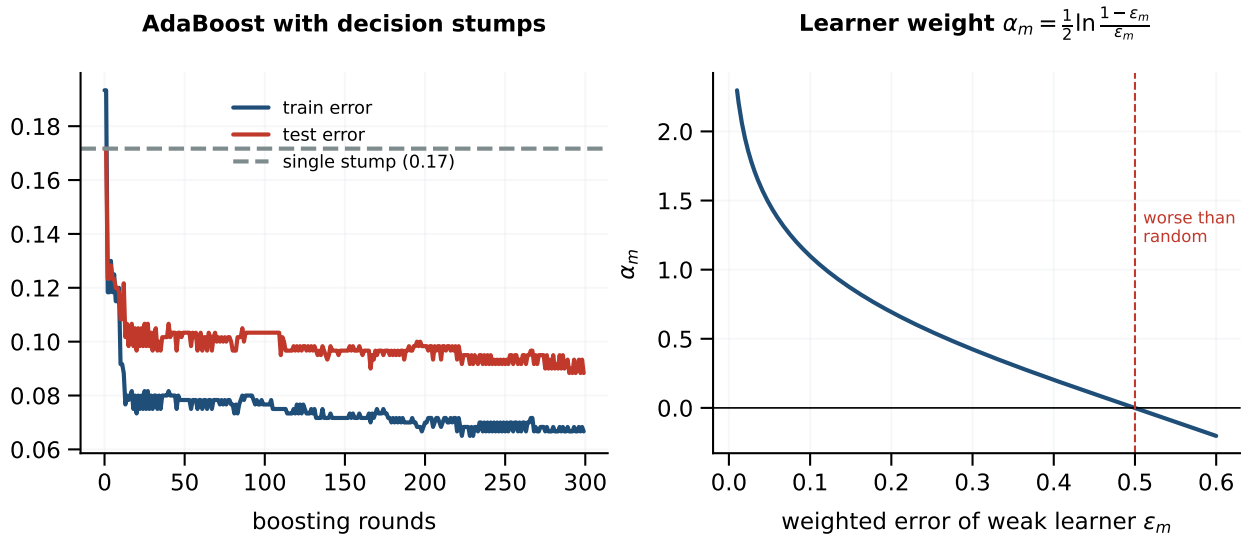


Figure 6.6: Left: boosting many stumps (each barely better than chance) drives the test error far below a single stump. Right: a learner with error near 0.5 gets almost no vote; one worse than random gets a negative weight.

Algorithm: Gradient boosting (Friedman, 2001)

1. Initialise with a constant: $F_0(\mathbf{x}) = \arg\min_\gamma \sum_i L(y_i, \gamma)$ (the mean for MSE, the log-odds for log loss).
2. For $m = 1, \ldots, M$:
   (a) Pseudo-residuals: $r_{im} = -\left[\dfrac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{m-1}}$.
   (b) Fit a regression tree h_m to $\{(\mathbf{x}_i, r_{im})\}$, giving leaf regions R_{jm}.
   (c) Leaf values: $\gamma_{jm} = \arg\min_\gamma \sum_{\mathbf{x}_i \in R_{jm}} L(y_i, F_{m-1}(\mathbf{x}_i) + \gamma)$.
   (d) Update with shrinkage ν: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu \sum_j \gamma_{jm} \mathbb{I}[\mathbf{x} \in R_{jm}]$.

Loss $L(y, F)$	Negative gradient (pseudo-residual)	Note
$\frac{1}{2}(y - F)^2$	$y - F$	ordinary residual
$\lvert y - F \rvert$	$\operatorname{sign}(y - F)$	robust
Log loss, $p = \sigma(F)$	$y - p$	classification
Huber	$y - F$ if $\lvert y - F \rvert \le \delta$, else $\delta \operatorname{sign}(y - F)$	robust & smooth

Regularisation knobs: learning rate ν (0.01–0.3), number of trees M (with early stopping), tree depth (3–8), `subsample`< 1 (stochastic gradient boosting), `min_samples_leaf`, feature subsampling.

6.7 XGBoost (Extreme Gradient Boosting)

XGBoost is gradient boosting with an explicitly regularised objective, a second-order (Newton) approximation, and systems engineering (sparsity-aware split finding, column blocks, parallelism, cache awareness). At round t:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} L(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t), \qquad \Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2, \tag{6.9}$$

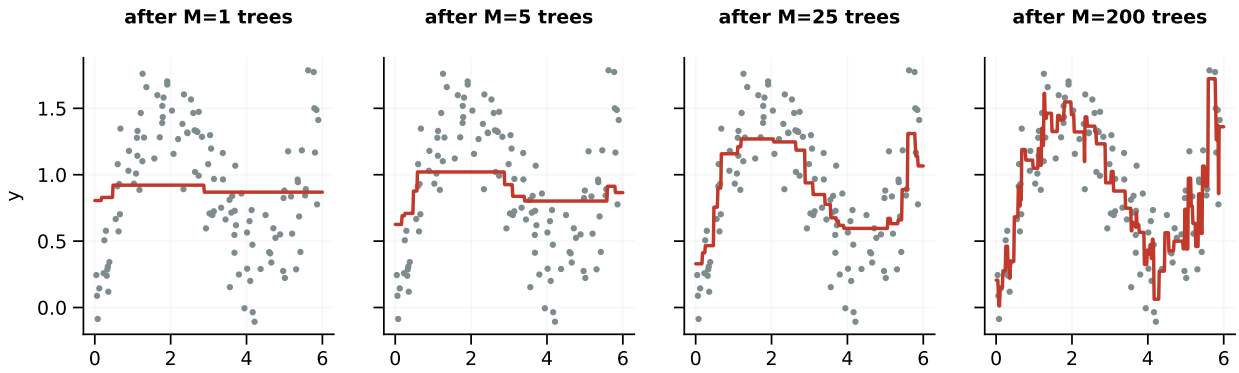


Figure 6.7: Gradient boosting with depth-2 trees ($\nu = 0.1$): the fit improves stage by stage, from a nearly constant prediction to a close fit of the signal.

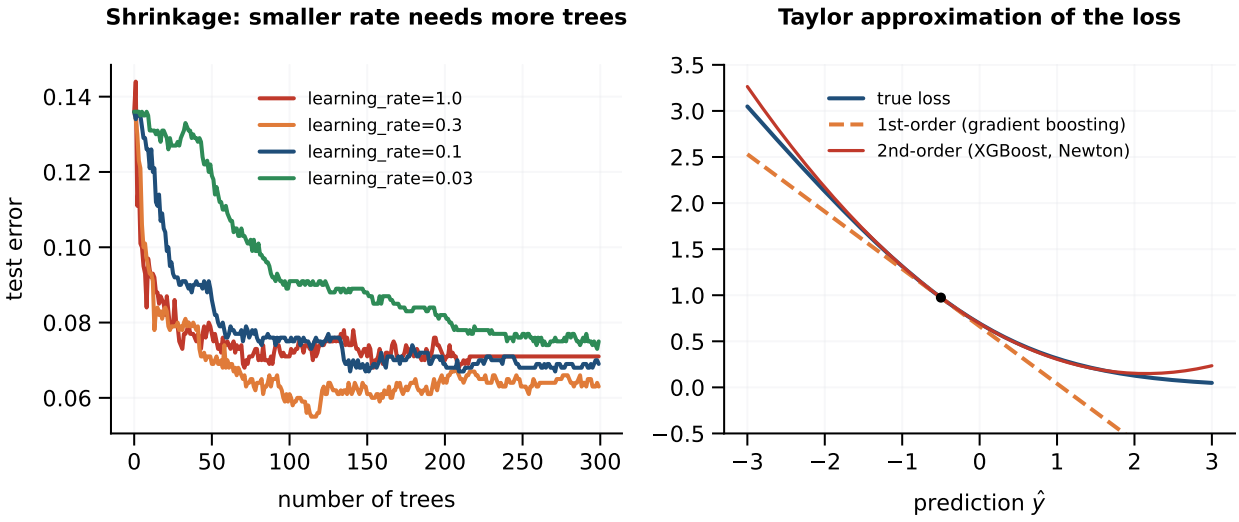


Figure 6.8: Left: a smaller learning rate needs more trees but usually reaches a lower error. Right: XGBoost replaces the first-order (linear) approximation of the loss with a second-order (quadratic) one, which gives closed-form leaf values.

with T leaves and leaf weights w_j. A second-order Taylor expansion with $g_i = \partial_{\hat{y}} L$ and $h_i = \partial^2_{\hat{y}} L$ at $\hat{y}_i^{(t-1)}$ gives

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^{n} \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \gamma T + \frac{1}{2} \lambda \sum_j w_j^2. \tag{6.10}$$

Grouping samples by leaf ($G_j = \sum_{i \in I_j} g_i$, $H_j = \sum_{i \in I_j} h_i$) makes each leaf an independent quadratic, so

$$\boxed{w_j^* = -\frac{G_j}{H_j + \lambda}}, \qquad \tilde{\mathcal{L}}^* = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T. \tag{6.11}$$

The **gain** of splitting a leaf into left/right children is

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma. \tag{6.12}$$

A split is kept only if the gain is positive, so γ acts as built-in pruning and λ shrinks leaf values. Other key parameters: `eta`, `max_depth`, `min_child_weight` (minimum H in a child), `subsample`, `colsample_bytree`, `reg_alpha` (L1 on leaf weights), `scale_pos_weight` (imbalance).

6.8 LightGBM and CatBoost

	XGBoost	LightGBM	CatBoost
Tree growth	Level-wise (depth-wise) by default	**Leaf-wise** (best-first): splits the leaf with the largest gain; control with `num_leaves`, `max_depth`	**Oblivious (symmetric)** trees: the same split for every node at a given depth
Speed tricks	Histogram / approximate splits, sparsity-aware	Histogram bins, **GOSS** (keep the top a% largest-gradient rows, sample b% of the rest, up-weight the sampled ones by $\frac{1-a}{b}$), **EFB** (bundle mutually exclusive sparse features)	Fast inference from symmetric trees
Categoricals	One-hot / native (recent versions)	Native categorical splits	**Best native support**: ordered target statistics
Special idea	Newton boosting + regularised objective	Very fast on large data	**Ordered boosting** reduces target leakage / prediction shift
Watch out	Many hyper-parameters	Leaf-wise can overfit small data	Slower training on CPU for large data

CatBoost's ordered target statistic for category value c of sample i uses only samples that come *before* i in a random permutation:

$$\hat{x}_i = \frac{\sum_{j<i} \mathbb{I}[x_j = c]\, y_j + a\, P}{\sum_{j<i} \mathbb{I}[x_j = c] + a}, \tag{6.13}$$

where P is a prior (e.g. the global mean) and $a > 0$ its weight – this prevents a sample's own label leaking into its encoding.

Task	Classification, regression, ranking
Data it fits	**Structured / tabular data** – the state of the art for most tabular problems; handles missing values, mixed types, non-linearities
Needs scaling?	No
Key hyper-parameters	`learning_rate`, `n_estimators` + early stopping, `max_depth` / `num_leaves`, `subsample`, `colsample_bytree`, `min_child_weight`, `reg_lambda`, `reg_alpha`
Strengths	Top accuracy on tabular data, flexible losses, handles missing values, feature importance and SHAP support
Weaknesses	Sequential training, sensitive to hyper-parameters, can overfit noisy data, less interpretable, poor extrapolation

Best use cases & data fit

Kaggle-style tabular competitions, credit risk, fraud, click-through and ranking (learning-to-rank objectives), demand forecasting with lag features, insurance pricing.

6.9 Stacking, blending and voting

- **Hard voting**: $\hat{y} = \text{mode}\{f_1(\mathbf{x}), \ldots, f_M(\mathbf{x})\}$. **Soft voting**: $\hat{y} = \arg\max_k \frac{1}{M} \sum_m P_m(y = k \mid \mathbf{x})$ – usually better when the models are calibrated.
- **Stacking**: train diverse base models; collect their *out-of-fold* predictions as new features; fit a meta-learner (often logistic/ridge regression) on them: $\hat{y} = g(f_1(\mathbf{x}), \ldots, f_M(\mathbf{x}))$. Out-of-fold predictions are essential – otherwise the meta-learner learns from overfitted in-sample predictions.
- **Blending**: like stacking but the meta-learner is trained on a single hold-out set (simpler, uses less data).

Ensembles gain most when the base models are *accurate and diverse* (different algorithms, features or data views).

CHAPTER 7

Unsupervised Learning

No labels: the goal is to discover structure – groups, low-dimensional representations, anomalies, or co-occurrence rules.

7.1 K-Means clustering

Intuition

Place k centres; assign every point to its nearest centre; move each centre to the mean of its points; repeat until nothing changes.

7.1.1 Objective (within-cluster sum of squares, WCSS / inertia)

$$J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2. \tag{7.1}$$

Algorithm: Lloyd's algorithm

1. Initialise K centroids (use **k-means++**: choose the first centre at random, then each next centre with probability $\propto D(\mathbf{x})^2$, the squared distance to the nearest centre already chosen).
2. **Assignment step**: $c_i = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$.
3. **Update step**: $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$.
4. Repeat 2–3 until assignments stop changing. Each step cannot increase J, so it converges – but only to a *local* minimum; run several initialisations (`n_init`).

Complexity $O(nKp)$ per iteration – scales to millions of points (Mini-Batch K-Means for more).

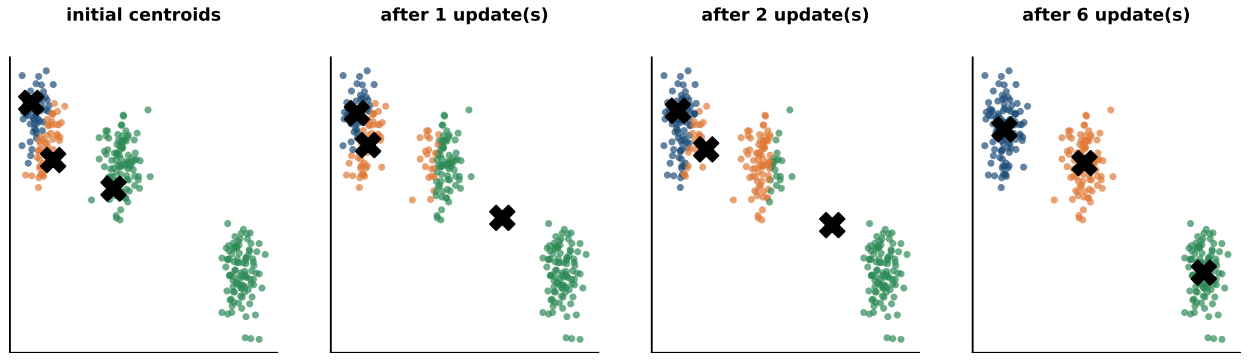


Figure 7.1: Lloyd's algorithm: starting from poor centroids, assignments and centroids converge after a few updates.

7.1.2 Choosing K

- **Elbow method**: plot J vs K; pick the "knee" where adding clusters stops helping much.
- **Silhouette score**: for point i, let $a(i)$ be its mean distance to its own cluster and $b(i)$ its mean distance to the nearest other cluster:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]. \tag{7.2}$$

Near 1 = well clustered, around 0 = on a border, negative = probably in the wrong cluster. Pick the K with the highest mean silhouette.

- Gap statistic, Davies–Bouldin and Calinski–Harabasz indices (Chapter 8), and above all *business usefulness*.

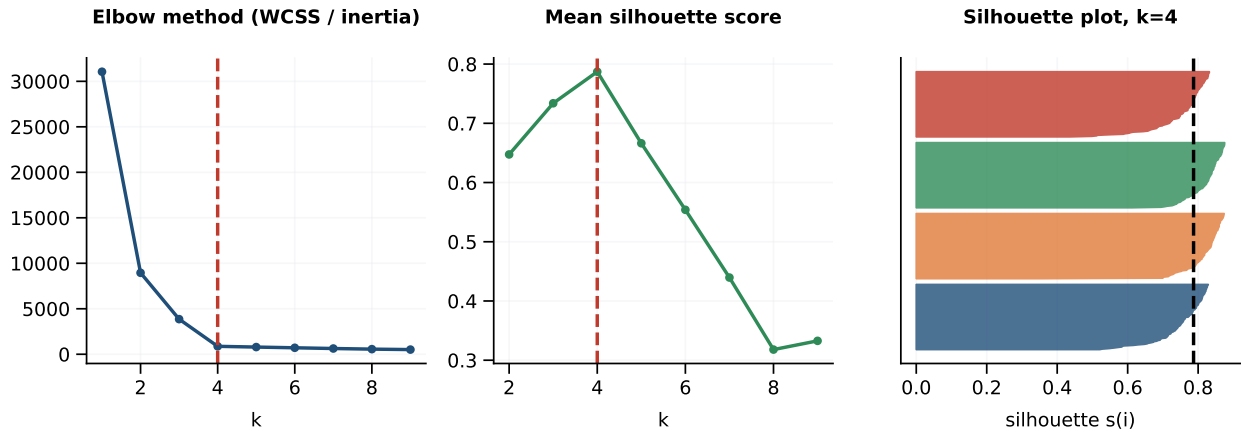


Figure 7.2: Elbow at $K = 4$, highest mean silhouette at $K = 4$, and per-cluster silhouette profiles.

Task	Clustering (hard assignment)
Data it fits	Numerical data with roughly spherical, similarly sized, well-separated clusters
Needs scaling?	**Yes** – Euclidean distance
Key hyper-parameters	`n_clusters`, `init`, `n_init`, `max_iter`
Strengths	Simple, fast, scalable, easy to interpret centroids
Weaknesses	Must choose K; assumes convex, isotropic clusters; sensitive to outliers and initialisation; categorical data need K-Modes / K-Prototypes

Best use cases & data fit

Customer segmentation, image colour quantisation, document grouping on embeddings, creating cluster features for supervised models, vector quantisation.

7.2 Hierarchical (agglomerative) clustering

Start with every point as its own cluster and repeatedly merge the two closest clusters; record the merges in a **dendrogram**. Cutting the dendrogram at a height gives a flat clustering – no need to fix K in advance.

7.2.1 Linkage criteria (distance between clusters A and B)

$$\text{Single:} \quad d(A,B) = \min_{a\in A, b\in B} d(a,b) \qquad \text{chains; finds elongated shapes} \tag{7.3}$$

$$\text{Complete:} \quad d(A,B) = \max_{a\in A, b\in B} d(a,b) \qquad \text{compact clusters} \tag{7.4}$$

$$\text{Average:} \quad d(A,B) = \frac{1}{|A||B|}\sum_{a\in A}\sum_{b\in B} d(a,b) \qquad \text{compromise} \tag{7.5}$$

$$\text{Ward:} \quad \Delta(A,B) = \frac{|A||B|}{|A|+|B|}\|\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\|^2 \qquad \text{smallest increase in WCSS} \tag{7.6}$$

Memory $O(n^2)$ and time $O(n^2 \log n)$ or worse ⇒ practical up to roughly 10^4 points.

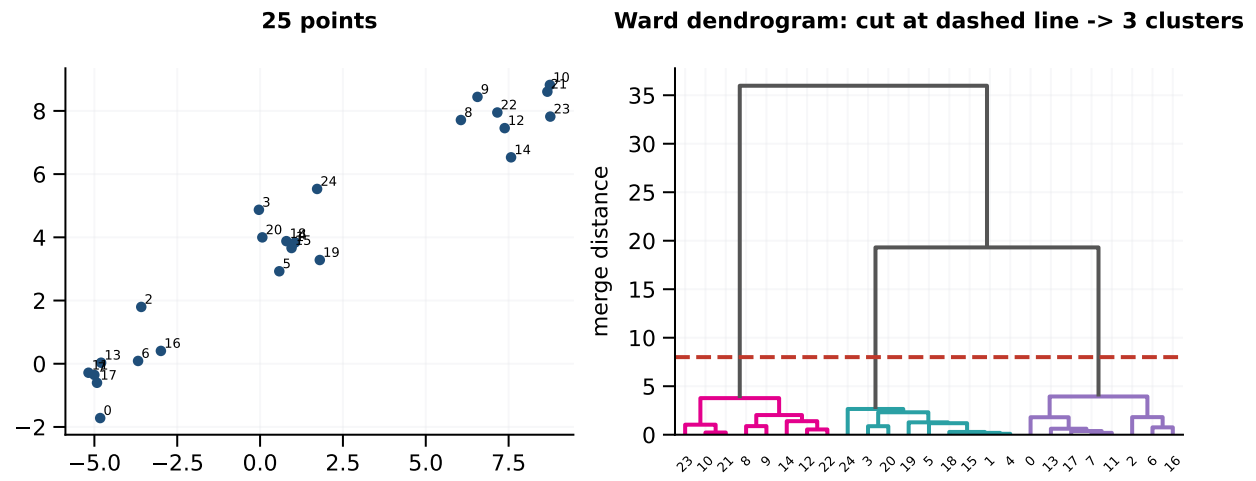


Figure 7.3: Ward dendrogram: the long vertical gap below the dashed line suggests three natural clusters.

7.3 DBSCAN (density-based clustering)

Intuition

Clusters are dense regions separated by sparse regions. A point with many neighbours is a core point; clusters grow by chaining core points; isolated points are labelled noise.

With radius ε and threshold `minPts`, let $N_\varepsilon(\mathbf{x}) = \{\mathbf{x}_j : d(\mathbf{x}, \mathbf{x}_j) \le \varepsilon\}$:

- **Core point**: $|N_\varepsilon(\mathbf{x})| \ge$ `minPts` (the point itself counts in scikit-learn).
- **Border point**: not core, but within ε of a core point.
- **Noise**: neither (label -1).

A cluster is a maximal set of density-connected points. Choose ε from the "knee" of the sorted k-distance plot ($k =$ `minPts`). **HDBSCAN** extends DBSCAN to clusters of varying density.

Task	Clustering + outlier detection
Data it fits	Arbitrary-shaped clusters, spatial/GPS data, data with noise
Needs scaling?	Yes
Key hyper-parameters	`eps`, `min_samples`, `metric`
Strengths	No need to set K; finds non-convex shapes; flags outliers
Weaknesses	Struggles with varying densities and high dimensions; sensitive to ε

7.4 Gaussian Mixture Models (GMM) and EM

A GMM is a soft, probabilistic generalisation of K-Means:

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \qquad \sum_k \pi_k = 1. \tag{7.7}$$

Algorithm: Expectation–Maximisation

E-step (responsibilities – soft assignments):

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

M-step (weighted re-estimation), with $N_k = \sum_i \gamma_{ik}$:

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_i \gamma_{ik} \mathbf{x}_i, \qquad \boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_i \gamma_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top, \qquad \pi_k = \frac{N_k}{n}.$$

Repeat until the log-likelihood $\sum_i \log \sum_k \pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ converges (it never decreases).

Choose K (and the covariance type: full, tied, diag, spherical) with information criteria, lower is better:

$$\text{AIC} = 2d - 2\ln\hat{\mathcal{L}}, \qquad \text{BIC} = d\ln n - 2\ln\hat{\mathcal{L}}, \tag{7.8}$$

where d is the number of free parameters. K-Means is the limit of a GMM with equal spherical covariances $\sigma^2\mathbf{I}$, $\sigma \to 0$ (hard assignments).

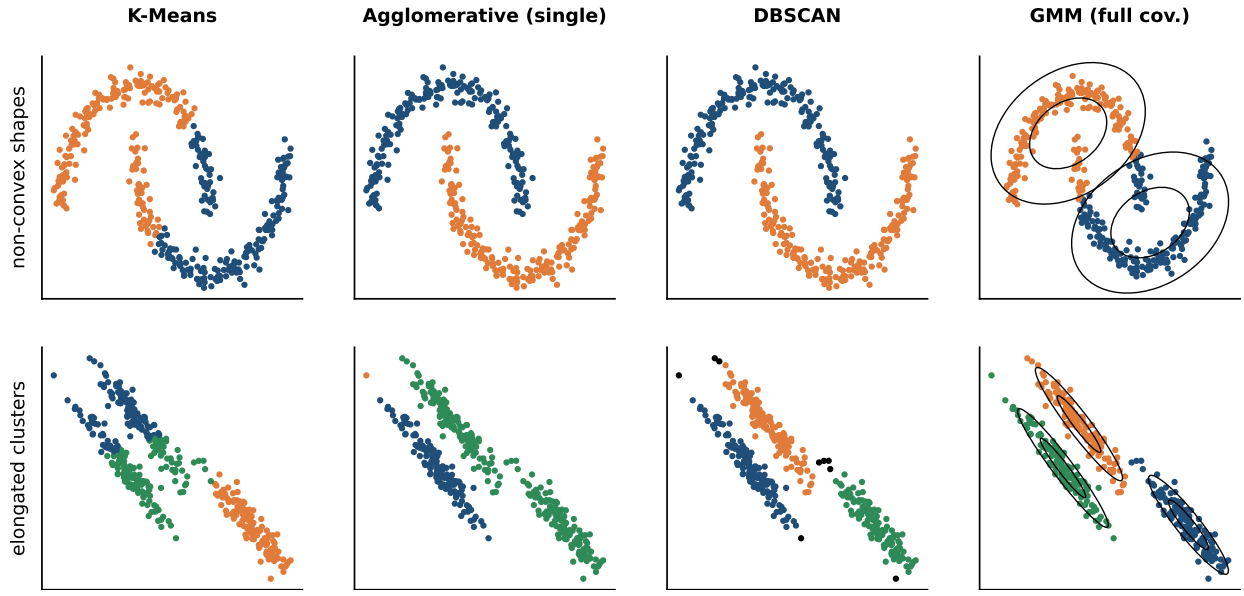


Figure 7.4: The same data through four algorithms. K-Means and GMM fail on the moons (top) but GMM's full covariances capture elongated clusters (bottom, ellipses = 1 and 2 standard deviations). DBSCAN and single linkage follow non-convex shapes; DBSCAN marks noise in black.

	K-Means	Hierarchical	DBSCAN	GMM
Need K?	Yes	No (cut tree)	No	Yes
Cluster shape	Spherical	Depends on linkage	Arbitrary	Ellipsoidal
Assignment	Hard	Hard	Hard + noise	Soft (probabilities)
Outliers	Sensitive	Sensitive	Detects them	Moderately sensitive
Scale	Large n	Small–medium n	Medium–large n	Medium n

7.5 Principal Component Analysis (PCA)

Intuition

Rotate the coordinate system so that the first axis points along the direction of greatest spread, the second along the next greatest (perpendicular to the first), and so on. Keep the first few axes and drop the rest.

7.5.1 Mathematics

Centre the data ($\mathbf{X}_c = \mathbf{X} - \bar{\mathbf{x}}$). The covariance matrix is

$$\mathbf{S} = \frac{1}{n-1}\mathbf{X}_c^\top \mathbf{X}_c. \tag{7.9}$$

The first principal direction maximises the projected variance:

$$\mathbf{v}_1 = \arg\max_{\|\mathbf{v}\|=1} \mathbf{v}^\top \mathbf{S}\mathbf{v} \quad \Longrightarrow \quad \mathbf{S}\mathbf{v}_1 = \lambda_1 \mathbf{v}_1 \tag{7.10}$$

(Lagrangian $\mathbf{v}^\top\mathbf{S}\mathbf{v} - \lambda(\mathbf{v}^\top\mathbf{v} - 1)$, set the derivative to zero). So the principal components are the **eigenvectors** of $\mathbf{S}$, and the eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots$ are the variances along them. In practice PCA is computed with the SVD $\mathbf{X}_c = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$: the columns of $\mathbf{V}$ are the components and $\lambda_k = \sigma_k^2/(n-1)$.

$$\text{Projection: } \mathbf{Z} = \mathbf{X}_c\mathbf{V}_k, \qquad \text{explained variance ratio}_k = \frac{\lambda_k}{\sum_j \lambda_j}, \qquad \text{reconstruction } \hat{\mathbf{X}} = \mathbf{Z}\mathbf{V}_k^\top + \bar{\mathbf{x}}. \tag{7.11}$$

Choose k to retain e.g. 90–95% of the variance (scree plot).

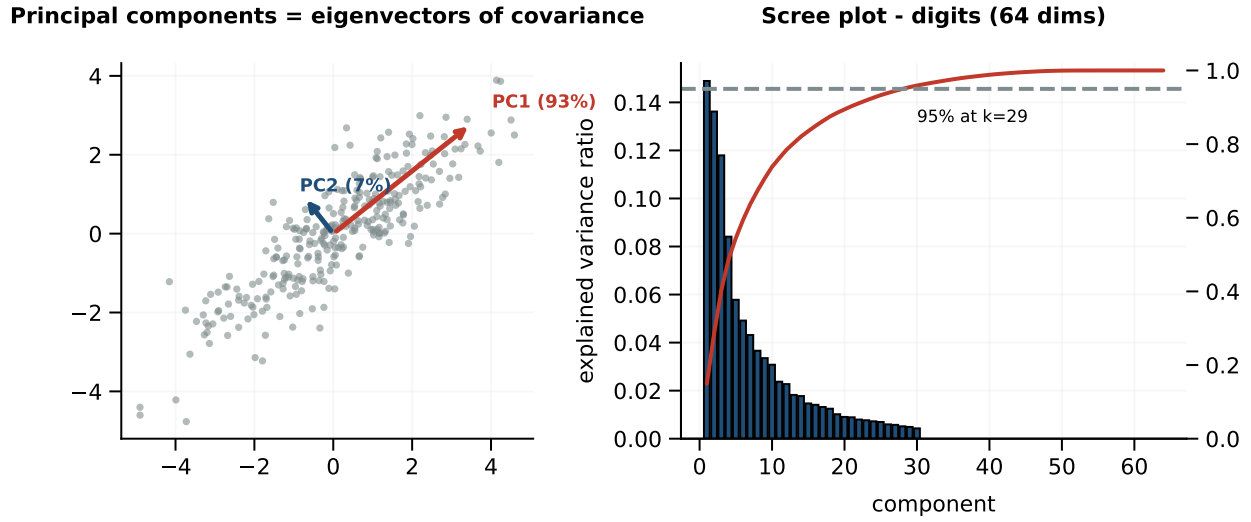


Figure 7.5: Left: PC1 and PC2 are the eigenvectors of the covariance matrix (arrow length $\propto$ standard deviation). Right: on 64-dimensional Digits, roughly 29 components retain 95% of the variance.

Task	Dimensionality reduction, compression, visualisation, de-noising
Data it fits	Numerical, correlated features; linear structure
Needs scaling?	**Yes** (standardise unless all features share units)
Key hyper-parameters	`n_components` (int or variance fraction), `whiten`
Strengths	Removes multicollinearity, speeds up downstream models, de-noises, deterministic
Weaknesses	Only linear structure; components are hard to interpret; variance $\neq$ predictive importance (unsupervised)

Variants: Kernel PCA (non-linear), Incremental PCA (big data), Sparse PCA (interpretable loadings), Truncated SVD / LSA (sparse text matrices, no centring).

7.6 t-SNE and UMAP (non-linear visualisation)

t-SNE converts distances into neighbour probabilities and matches them in 2-D:

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2/2\sigma_i^2)}{\sum_{k\neq i}\exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2/2\sigma_i^2)}, \qquad p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}, \tag{7.12}$$

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k\neq l}(1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}, \qquad \min_{\mathbf{Y}} \ \mathrm{KL}(P\|Q) = \sum_{i\neq j} p_{ij}\log\frac{p_{ij}}{q_{ij}}. \tag{7.13}$$

σ_i is set so each point's neighbourhood has a target *perplexity* (typically 5–50). The heavy-tailed Student-t in the low-dimensional space prevents crowding. **UMAP** builds a fuzzy k-NN graph and optimises a cross-entropy between graphs; it is faster, preserves more global structure and can transform new data.

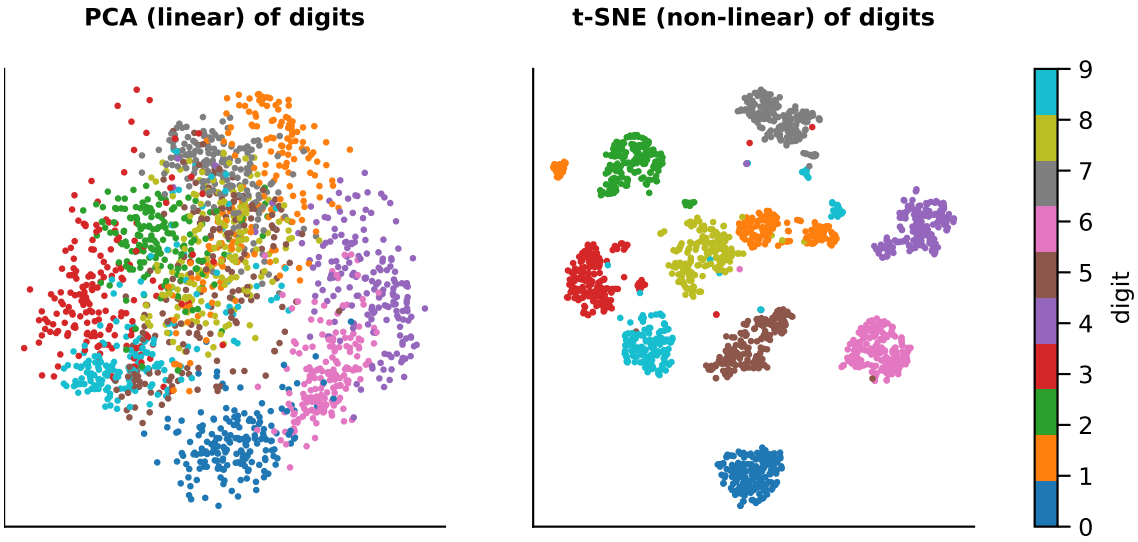


Figure 7.6: PCA vs t-SNE on the Digits data: t-SNE separates the ten digits into clear groups.

Pitfalls & gotchas

t-SNE is for *visualisation only*: cluster sizes and distances between clusters in the plot are not meaningful, results depend on perplexity and random seed, and it has no `transform` for new data.

7.7 Anomaly (outlier) detection

Method	Idea / formula	Good for
Z-score	$\lvert z\rvert = \lvert x-\mu\rvert/\sigma > 3$	Univariate, roughly normal
IQR rule	outside $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$	Univariate, skewed
Mahalanobis	$D^2 = (\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})$	Multivariate Gaussian data
Isolation Forest	Anomalies are isolated by fewer random splits	High-dim tabular, fast
Local Outlier Factor	Local density relative to neighbours' density	Clusters of varying density
One-Class SVM	Boundary enclosing most of the data (RBF)	Novelty detection, small data
Autoencoder	High reconstruction error	Images, sensors, complex data

Isolation Forest score. With $h(\mathbf{x})$ the path length to isolate $\mathbf{x}$ in a random tree and $c(n)$ the average path length of an unsuccessful BST search,

$$s(\mathbf{x}, n) = 2^{-\frac{\mathbb{E}[h(\mathbf{x})]}{c(n)}}, \qquad c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \quad H(i) \approx \ln i + 0.5772. \tag{7.14}$$

$s \to 1$: anomaly; $s \ll 0.5$: normal.

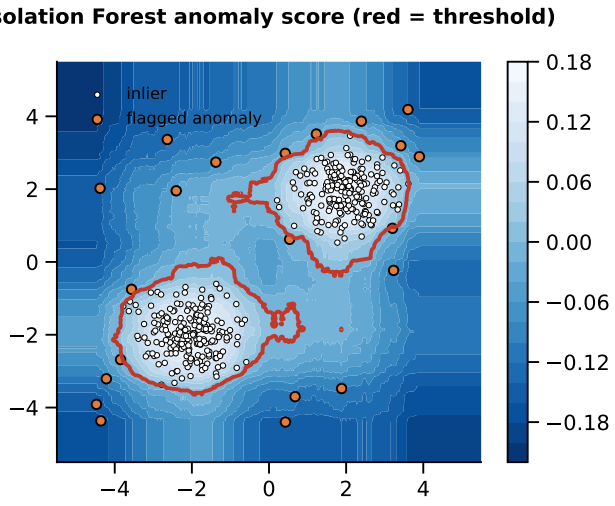


Figure 7.7: Isolation Forest scores: points far from the two dense blobs are isolated quickly and flagged.

7.8 Association rule mining (market-basket analysis)

For itemsets X and Y over N transactions:

$$\mathrm{support}(X) = \frac{\#\{\text{transactions containing } X\}}{N}, \tag{7.15}$$

$$\mathrm{confidence}(X \Rightarrow Y) = \frac{\mathrm{support}(X \cup Y)}{\mathrm{support}(X)} = \hat{P}(Y \mid X), \tag{7.16}$$

$$\mathrm{lift}(X \Rightarrow Y) = \frac{\mathrm{confidence}(X \Rightarrow Y)}{\mathrm{support}(Y)} \quad (>1:\ \text{positive association};\ =1:\ \text{independent}). \tag{7.17}$$

Apriori uses the anti-monotone property (every subset of a frequent itemset is frequent) to prune the search; **FP-Growth** avoids candidate generation with a prefix tree. *Example*: 1,000 baskets,

bread in 200, butter in 150, both in 100: support= 0.10, confidence(bread⇒butter)= 0.50, lift= 0.50/0.15 = 3.33.

CHAPTER 8

Evaluation Metrics

The metric encodes what "good" means for the business. Choose it *before* modelling.

8.1 Regression metrics

With residuals $e_i = y_i - \hat{y}_i$:

$$\text{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i| \qquad \text{same units as } y;\ \text{robust to outliers} \tag{8.1}$$

$$\text{MSE} = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2 \qquad \text{penalises large errors heavily} \tag{8.2}$$

$$\text{RMSE} = \sqrt{\text{MSE}} \qquad \text{same units as } y \tag{8.3}$$

$$\text{MAPE} = \frac{100\%}{n}\sum_i \left|\frac{y_i - \hat{y}_i}{y_i}\right| \qquad \text{scale-free; undefined at } y_i = 0 \tag{8.4}$$

$$\text{RMSLE} = \sqrt{\frac{1}{n}\sum_i \big(\log(1+\hat{y}_i) - \log(1+y_i)\big)^2} \qquad \text{relative errors; skewed positive targets} \tag{8.5}$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \qquad \text{share of variance explained} \tag{8.6}$$

$$R^2_{\text{adj}} = 1 - (1 - R^2)\frac{n-1}{n-p-1} \qquad \text{penalises useless features} \tag{8.7}$$

$R^2 = 1$ is perfect, $R^2 = 0$ equals predicting the mean, and $R^2 < 0$ is worse than the mean (possible on test data). Plain R^2 never decreases when features are added; adjusted R^2 can.

Choose	When
MAE	Outliers exist and all errors cost proportionally (e.g. delivery minutes)
RMSE	Large errors are disproportionately costly (e.g. energy load, safety)
MAPE / sMAPE	Business wants percentage errors and y is far from 0 (sales forecasts)
RMSLE	Target spans orders of magnitude; under-prediction should cost more than over-prediction
R^2 / **adj.** R^2	Explaining variance, comparing models on the same data

8.2 Classification metrics

8.2.1 The confusion matrix

	Predicted positive	Predicted negative
Actual positive	**TP** (true positive)	**FN** (false negative, Type II error)
Actual negative	**FP** (false positive, Type I error)	**TN** (true negative)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{8.8}$$

$$\text{Precision (PPV)} = \frac{TP}{TP + FP} \tag{8.9}$$

$$\text{Recall (sensitivity, TPR)} = \frac{TP}{TP + FN} \tag{8.10}$$

$$\text{Specificity (TNR)} = \frac{TN}{TN + FP} \tag{8.11}$$

$$\text{False positive rate (FPR)} = \frac{FP}{FP + TN} = 1 - \text{specificity} \tag{8.12}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{8.13}$$

$$F_\beta = (1 + \beta^2) \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \,\text{Precision} + \text{Recall}} \tag{8.14}$$

$$\text{Balanced accuracy} = \tfrac{1}{2}(\text{TPR} + \text{TNR}) \tag{8.15}$$

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{8.16}$$

$$\text{Cohen's } \kappa = \frac{p_o - p_e}{1 - p_e} \tag{8.17}$$

In words: *precision* is the share of predicted positives that are correct; *recall* is the share of actual positives that were caught; F_1 is their harmonic mean (it is low if either is low); F_β with $\beta > 1$ weights recall more. MCC ranges over $[-1, 1]$ and uses all four cells, so it stays honest under imbalance. Cohen's κ measures agreement beyond chance, where p_o is observed accuracy and p_e is the accuracy expected from the marginal totals.

Worked example (imbalanced: 120 positives in 1,000)

$TP = 80,\ FP = 20,\ FN = 40,\ TN = 860.$
Accuracy $= 940/1000 = 0.940$ Precision $= 80/100 = 0.800$ Recall $= 80/120 = 0.667$ Specificity $= 860/880 = 0.977$
$F_1 = 2(0.8)(0.667)/(1.467) = 0.727$ $F_2 = 0.690$ Balanced accuracy $= 0.822$
MCC $= (80 \cdot 860 - 20 \cdot 40)/\sqrt{100 \cdot 120 \cdot 880 \cdot 900} = 68000/97488 \approx 0.698$
Cohen's κ: $p_o = 0.94$, $p_e = \frac{100 \cdot 120 + 900 \cdot 880}{1000^2} = 0.804$, $\kappa = (0.94 - 0.804)/(0.196) \approx 0.694$.
Lesson: a model that always predicts "negative" would score accuracy 0.88 but recall 0 and MCC 0 – accuracy alone is misleading on imbalanced data.

8.2.2 Threshold-free metrics: ROC-AUC and PR-AUC

- **ROC curve**: TPR vs FPR as the threshold moves from 1 to 0. **AUC** equals the probability that a randomly chosen positive is scored higher than a randomly chosen negative; 0.5 = random, 1.0 = perfect ranking. Insensitive to class imbalance – which can make it look optimistic when positives are rare.
- **Precision–Recall curve**: precision vs recall; the baseline is the positive prevalence. **Average precision (AP)** $= \sum_n (R_n - R_{n-1}) P_n$ summarises it. Prefer PR-AUC when positives are rare and you care about them (fraud, disease).

8.2.3 Probabilistic metrics

$$\text{Log loss} = -\frac{1}{n} \sum_i \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] \tag{8.18}$$

$$\text{Brier score} = \frac{1}{n} \sum_i (\hat{p}_i - y_i)^2 \tag{8.19}$$

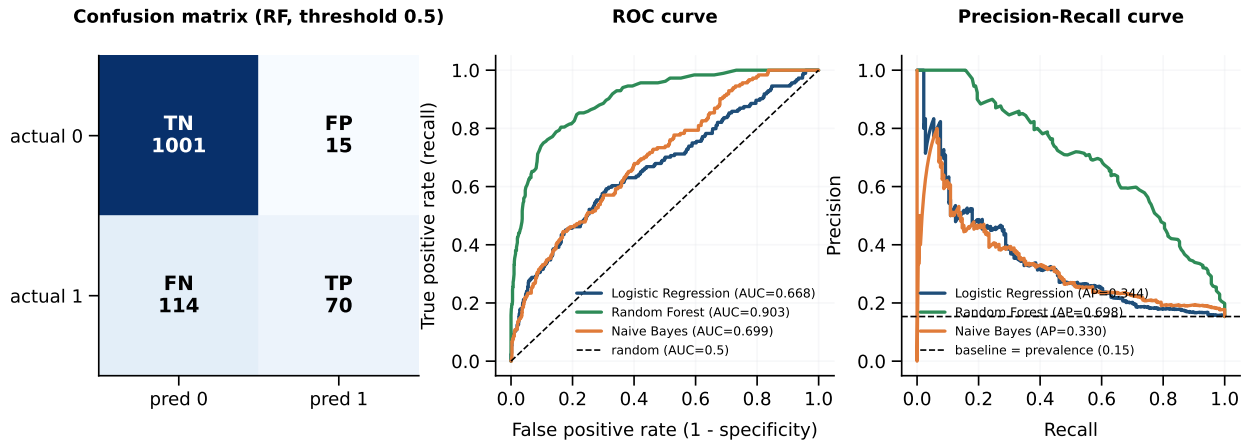


Figure 8.1: Confusion matrix, ROC and PR curves for three models on a 15%-positive problem. AUC values look similar and high; the PR curve separates the models more clearly.

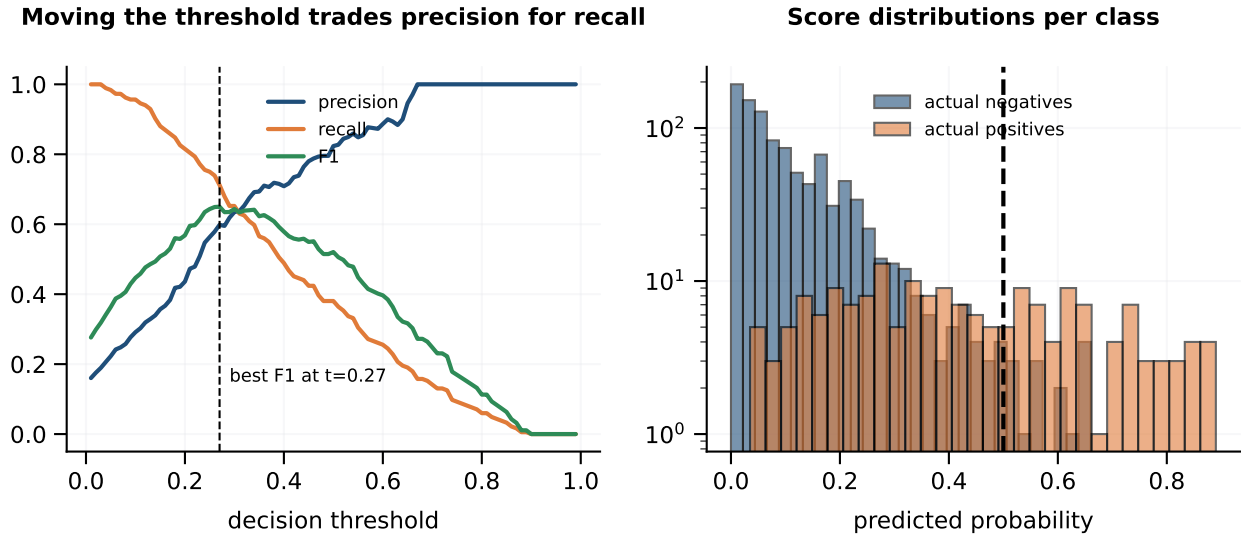


Figure 8.2: Left: raising the threshold increases precision and lowers recall; choose it from the business cost of FP vs FN (or maximise F_1). Right: the threshold cuts through the predicted-score distributions of the two classes.

Both reward *calibrated* probabilities. A model is calibrated if, among cases scored 0.7, about 70% are positive (check with a reliability diagram; fix with Platt/sigmoid or isotonic calibration).

8.2.4 Multiclass averaging

- **Macro**: compute the metric per class, then take the unweighted mean – every class counts equally (good for imbalance).
- **Weighted**: per-class mean weighted by class support.
- **Micro**: pool all TP/FP/FN globally (for single-label multiclass, micro-F_1 = accuracy).

8.2.5 Which classification metric?

Situation	Metric
Balanced classes, equal costs	Accuracy, F_1, ROC-AUC
False negatives are costly	Recall, F_2 (cancer screening, fraud, safety defects)
False positives are costly	Precision, $F_{0.5}$ (spam filter, legal flags, costly manual reviews)
Rare positive class	PR-AUC / AP, MCC, balanced accuracy, recall at fixed precision
Need ranking quality	ROC-AUC (credit scoring, Gini $= 2 \cdot \text{AUC} - 1$)
Need good probabilities	Log loss, Brier score, calibration curve
Unequal business costs	Expected cost $= c_{FP} \cdot FP + c_{FN} \cdot FN$; pick the threshold that minimises it

8.3 Clustering metrics

Internal (no labels):

$$\text{Silhouette} = \frac{1}{n}\sum_{i}\frac{b(i)-a(i)}{\max\{a(i),b(i)\}} \qquad \text{higher is better, } [-1,1] \tag{8.20}$$

$$\text{Davies–Bouldin} = \frac{1}{K}\sum_{i=1}^{K}\max_{j\neq i}\frac{s_i+s_j}{d(\boldsymbol{\mu}_i,\boldsymbol{\mu}_j)} \qquad \text{lower is better} \tag{8.21}$$

$$\text{Calinski–Harabasz} = \frac{\operatorname{tr}(\mathbf{B}_K)/(K-1)}{\operatorname{tr}(\mathbf{W}_K)/(n-K)} \qquad \text{higher is better} \tag{8.22}$$

(s_i = average distance of points in cluster i to its centroid; $\mathbf{B}_K, \mathbf{W}_K$ = between- and within-cluster scatter matrices.)

External (ground-truth labels available): Adjusted Rand Index (ARI; 0 = random labelling, 1 = perfect), Normalised Mutual Information (NMI), homogeneity, completeness and V-measure.

CHAPTER 9

Overfitting, Regularisation & Cross-Validation

9.1 Underfitting and overfitting

	Underfitting	Good fit	Overfitting
Training error	High	Low	Very low
Validation error	High (close to training)	Low (close to training)	High (large gap)
Cause	Model too simple, weak features, too much regularisation	Right complexity	Model too complex, too little data, noise memorised, leakage
Bias / variance	High bias	Balanced	High variance
Fixes	More complex model, more/better features, less regularisation, train longer	–	More data, regularisation, simpler model, feature selection, early stopping, bagging, dropout, augmentation

(See Figure 4.2 for the classic polynomial example.)

9.2 The bias–variance decomposition

Assume $y = f(\mathbf{x}) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\mathrm{Var}(\varepsilon) = \sigma^2$, and let $\hat{f}$ be trained on a random training set. At a point $\mathbf{x}$, the expected squared test error decomposes as

$$\mathbb{E}\big[(y - \hat{f}(\mathbf{x}))^2\big] = \underbrace{\big(f(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})]\big)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\big[(\hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})])^2\big]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible noise}}. \tag{9.1}$$

Derivation sketch: write $y - \hat{f} = (f - \mathbb{E}\hat{f}) + (\mathbb{E}\hat{f} - \hat{f}) + \varepsilon$, square, and take expectations; all cross terms vanish because $\mathbb{E}[\hat{f} - \mathbb{E}\hat{f}] = 0$ and ε is independent of $\hat{f}$.

- **Bias**: error from wrong assumptions (a line for a curve). Simple models ⇒ high bias.
- **Variance**: sensitivity to the particular training sample. Flexible models ⇒ high variance.
- Noise σ^2 cannot be removed by any model.

9.2.1 Diagnosing with learning and validation curves

- **Learning curve** (score vs training-set size): if training and validation curves converge at a *low* score ⇒ high bias (more data will not help; use a richer model). If there is a large gap that narrows as data grow ⇒ high variance (more data or regularisation will help).
- **Validation curve** (score vs one hyper-parameter): shows the underfit → sweet spot → overfit transition.

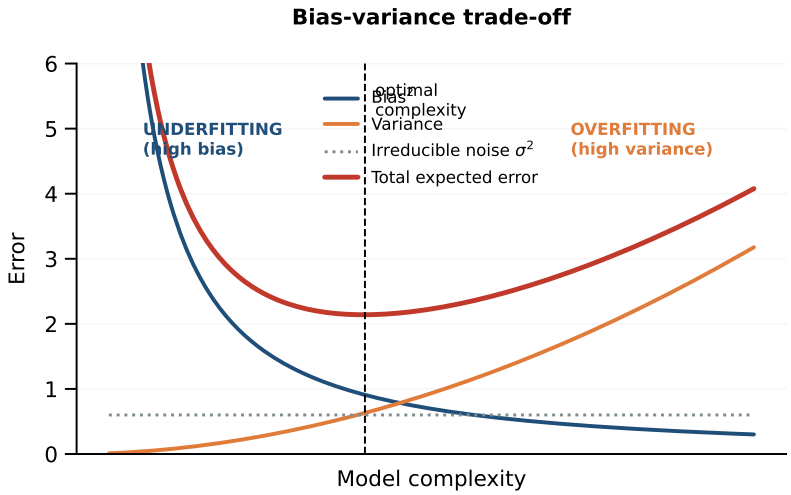


Figure 9.1: As complexity grows, bias falls and variance rises; total error is U-shaped.

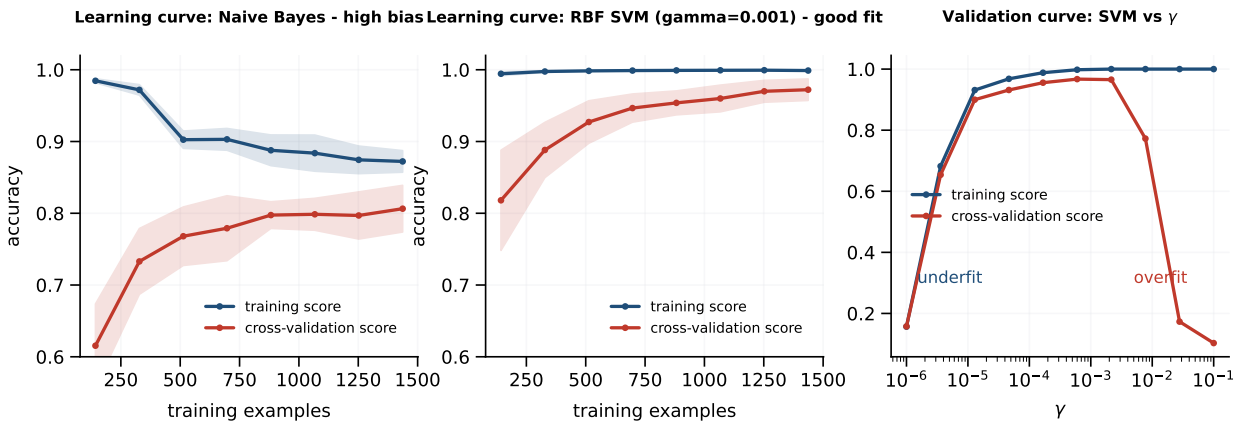


Figure 9.2: Left: Naive Bayes plateaus at a lower score (bias). Middle: an RBF SVM reaches a high score with a small gap. Right: as γ grows the training score keeps rising while the CV score collapses – overfitting.

9.3 Regularisation techniques

Regularisation adds information or constraints that favour simpler models, trading a little bias for a large reduction in variance.

9.3.1 L2 (ridge / weight decay)

$$J_{\text{reg}}(\mathbf{w}) = J(\mathbf{w}) + \lambda\|\mathbf{w}\|_2^2, \qquad \nabla J_{\text{reg}} = \nabla J + 2\lambda\mathbf{w} \;\Rightarrow\; \mathbf{w} \leftarrow (1 - 2\eta\lambda)\mathbf{w} - \eta\nabla J. \tag{9.2}$$

Each step first "decays" the weights towards zero. Smooth shrinkage, keeps every feature, stabilises correlated features.

9.3.2 L1 (lasso)

$$J_{\text{reg}}(\mathbf{w}) = J(\mathbf{w}) + \lambda\|\mathbf{w}\|_1, \qquad \partial|w_j| = \begin{cases} \text{sign}(w_j) & w_j \neq 0 \\ [-1, 1] & w_j = 0 \end{cases} \tag{9.3}$$

The constant-size push towards zero (independent of $|w_j|$) drives small weights exactly to zero $\Rightarrow$ sparse models and feature selection.

9.3.3 Geometric view

The penalised problem is equivalent to minimising the loss subject to $\|\mathbf{w}\|_q \leq t$. Loss contours (ellipses) first touch the L1 diamond at a *corner*, where some coordinates are zero; they touch the L2 circle at a generic point where all coordinates are non-zero.

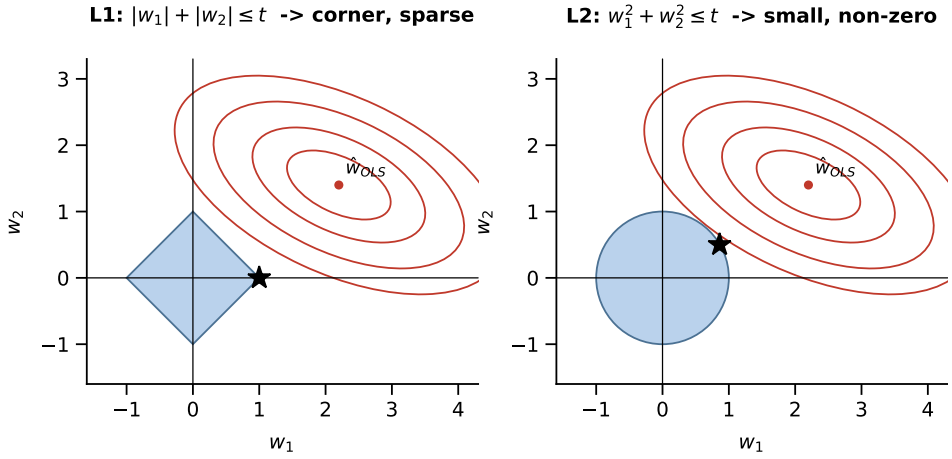


Figure 9.3: Why L1 gives sparse solutions and L2 does not.

9.3.4 Bayesian view

Regularised least squares is *maximum a posteriori* (MAP) estimation:

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg\max_{\mathbf{w}} \; \log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) + \log p(\mathbf{w}). \tag{9.4}$$

A Gaussian prior $w_j \sim \mathcal{N}(0, \tau^2)$ gives L2 with $\lambda = \sigma^2/\tau^2$; a Laplace prior $p(w_j) \propto e^{-|w_j|/b}$ gives L1. Strong regularisation = strong prior belief that weights are small.

9.3.5 Other regularisers

Technique	How it regularises
Elastic Net	$\alpha\rho\|\mathbf{w}\|_1 + \frac{\alpha(1-\rho)}{2}\|\mathbf{w}\|_2^2$: sparsity + grouping of correlated features
Early stopping	Stop training when validation loss stops improving for `patience` rounds (GBMs, neural nets); limits effective model capacity
Dropout	In training, zero each hidden unit with probability p and scale the survivors by $1/(1-p)$ (inverted dropout); approximately averages an ensemble of thinned networks
Tree constraints	`max_depth`, `min_samples_leaf`, `ccp_alpha` pruning; in XGBoost $\gamma T + \frac{1}{2}\lambda \sum w_j^2$
Shrinkage & subsampling	Learning rate ν, row/column subsampling in boosting and random forests
Data augmentation	Enlarge training data with label-preserving transformations (flips, crops, noise, synonym swaps)
Noise injection, label smoothing	Make the model robust to small perturbations; avoid over-confident outputs
Batch normalisation, max-norm	Stabilise and constrain neural-network weights
Dimensionality reduction	PCA / feature selection reduce capacity

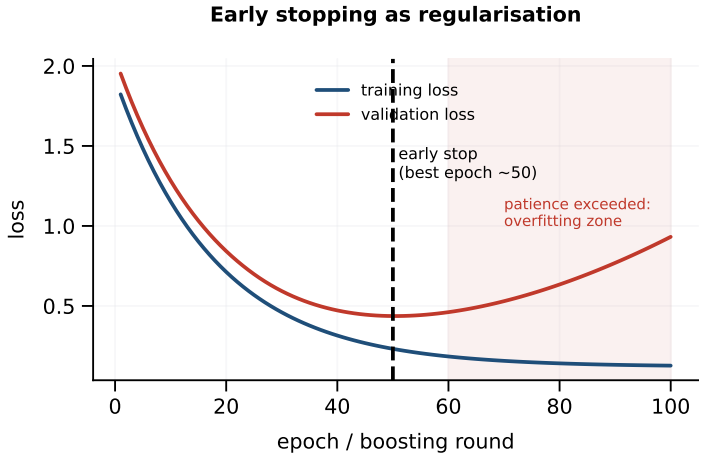


Figure 9.4: Early stopping: training loss keeps falling while validation loss turns upward; keep the model from the best validation epoch.

Pitfalls & gotchas

Always scale features before L1/L2 regularisation; otherwise the penalty depends on the units of each feature. In scikit-learn, `alpha` means *more* regularisation when larger, but `C` (logistic regression, SVM) means *less* regularisation when larger.

9.4 Cross-validation

Intuition

One train/validation split gives a noisy estimate that depends on which rows happened to land in validation. Cross-validation rotates the validation role across the data and averages the scores.

9.4.1 K-fold cross-validation

Split the data into K folds; for $k = 1, \ldots, K$ train on $K-1$ folds and evaluate on fold k:

$$\mathrm{CV}_{(K)} = \frac{1}{K}\sum_{k=1}^{K} \mathrm{Err}_k, \qquad \widehat{\mathrm{SE}} = \frac{\mathrm{sd}(\mathrm{Err}_1, \ldots, \mathrm{Err}_K)}{\sqrt{K}}. \tag{9.5}$$

Report mean $\pm$ standard deviation. $K = 5$ or 10 is the usual bias–variance compromise for the estimate itself: small K trains on less data (pessimistic bias); LOOCV ($K = n$) has low bias but high variance and is expensive. **One-standard-error rule**: choose the simplest model whose CV error is within one standard error of the best.

9.4.2 Choosing the right scheme

Scheme	Use when
Hold-out (train/val/test)	Very large datasets; quick iteration
K-Fold	Default for regression on i.i.d. data
Stratified K-Fold	Classification – preserves class ratios in every fold (essential with imbalance)
Repeated (Stratified) K-Fold	Small data; reduce variance of the estimate by repeating with different shuffles
Leave-One-Out (LOOCV)	Very small datasets
Group K-Fold	Several rows per entity (patient, customer, meter); no group in both train and validation
TimeSeriesSplit / walk-forward	Temporal data: train on the past, validate on the future, optional gap
Nested CV	Unbiased performance estimate *while* tuning: inner loop tunes, outer loop evaluates

Pitfalls & gotchas

- **Preprocessing inside CV**: scaling, imputation, feature selection, target encoding and SMOTE must be fitted inside each training fold – put them in a `Pipeline` and pass the pipeline to `cross_val_score` / `GridSearchCV`.
- The CV score of the *best* configuration after tuning is optimistically biased; confirm on an untouched test set or use nested CV.
- Never shuffle time-series data before splitting.

9.5 Hyper-parameter tuning

Parameters are learned from data (weights, split thresholds); **hyper-parameters** are set before training (depth, C, λ, learning rate) and tuned on validation folds.

- **Grid search**: try every combination. Exhaustive but exponential in the number of hyper-parameters.
- **Random search**: sample configurations from distributions (log-uniform for C, λ, learning rate). With the same budget it explores many more distinct values of the important hyper-parameters (Figure 9.6); usually more efficient than grid search.
- **Bayesian optimisation** (Optuna/TPE, Gaussian processes): fit a surrogate model of score vs hyper-parameters and pick the next trial with an acquisition function, e.g. Expected Improvement

$$\mathrm{EI}(\boldsymbol{\theta}) = \mathbb{E}\big[\max\left(f(\boldsymbol{\theta}) - f(\boldsymbol{\theta}^+), 0\right)\big] = (\mu - f^+)\,\Phi(Z) + \sigma\,\phi(Z), \qquad Z = \frac{\mu - f^+}{\sigma}, \tag{9.6}$$

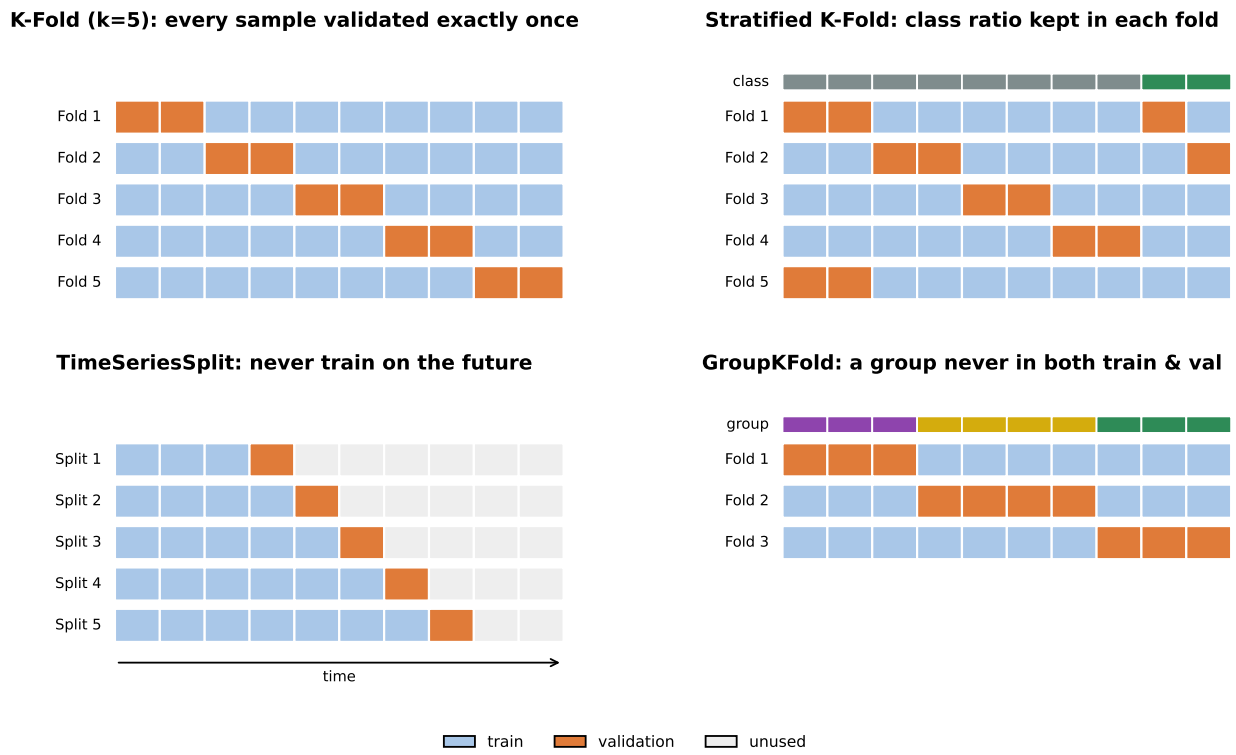


Figure 9.5: Cross-validation schemes. Blue = training rows, orange = validation rows.

where μ, σ are the surrogate's mean and standard deviation, f^+ is the best score so far and Φ, ϕ are the standard normal CDF and PDF.

- **Successive halving / Hyperband**: start many configurations on a small budget and keep only the best fraction at each round (`HalvingGridSearchCV`).

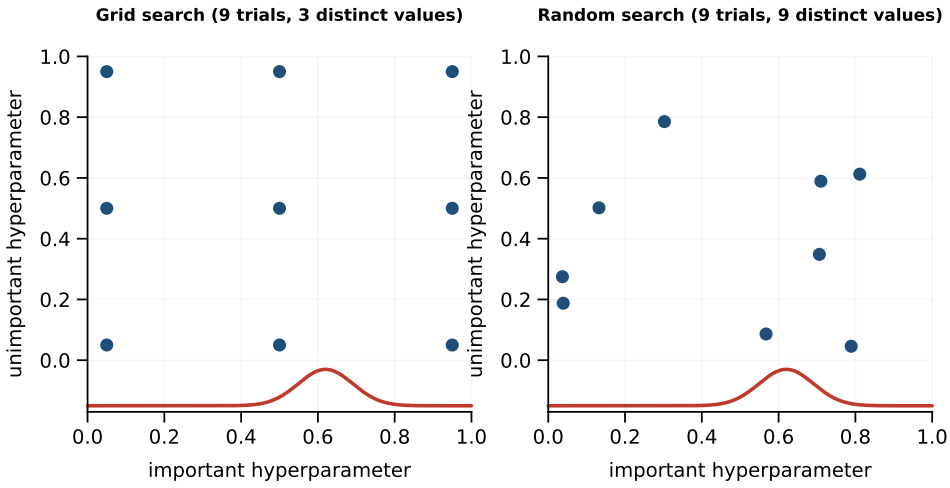


Figure 9.6: With nine trials, grid search tests only three values of the important hyper-parameter; random search tests nine.

9.6 Imbalanced data

- **Use the right metric**: PR-AUC, recall, F_β, MCC – not accuracy.
- **Class weights**: weight the loss by class; scikit-learn's `class_weight="balanced"` uses $w_c =$

$\frac{n}{K \cdot n_c}$. XGBoost: `scale_pos_weight`$\approx n_{\text{neg}}/n_{\text{pos}}$.

- **Resampling** (training folds only): random under-sampling of the majority class, random over-sampling of the minority class, or **SMOTE**, which creates synthetic minority points on segments between a minority sample and one of its k minority neighbours:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda\,(\mathbf{x}_{nn} - \mathbf{x}_i), \qquad \lambda \sim U(0,1). \tag{9.7}$$

  Variants: Borderline-SMOTE, ADASYN, SMOTE-Tomek, SMOTE-NC (categorical).

- **Threshold moving**: keep the model, but choose the decision threshold on validation data to maximise F_β or minimise expected cost.
- **Algorithms**: balanced random forest, boosting with weights, anomaly-detection framing for extreme imbalance.
- **Focal loss** (deep learning): $\text{FL}(p_t) = -\alpha_t(1-p_t)^\gamma \log p_t$ down-weights easy examples.

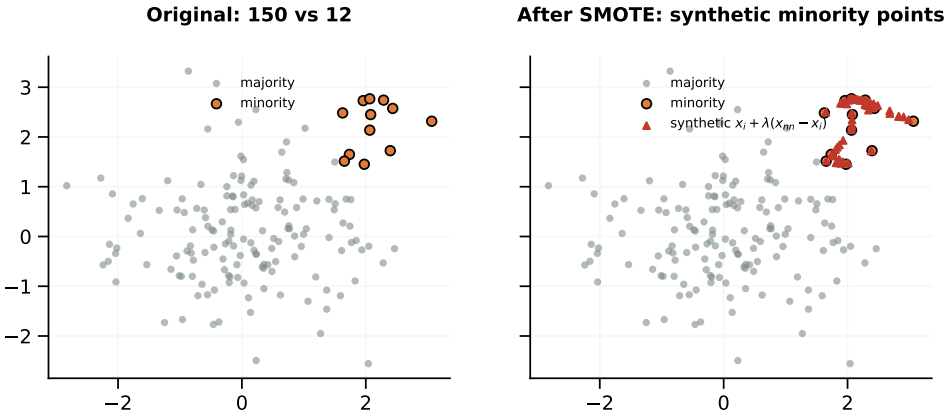


Figure 9.7: SMOTE interpolates new minority samples between existing minority neighbours.

CHAPTER 10

Choosing the Right Model: Cheat Sheets

10.1 Same data, nine models

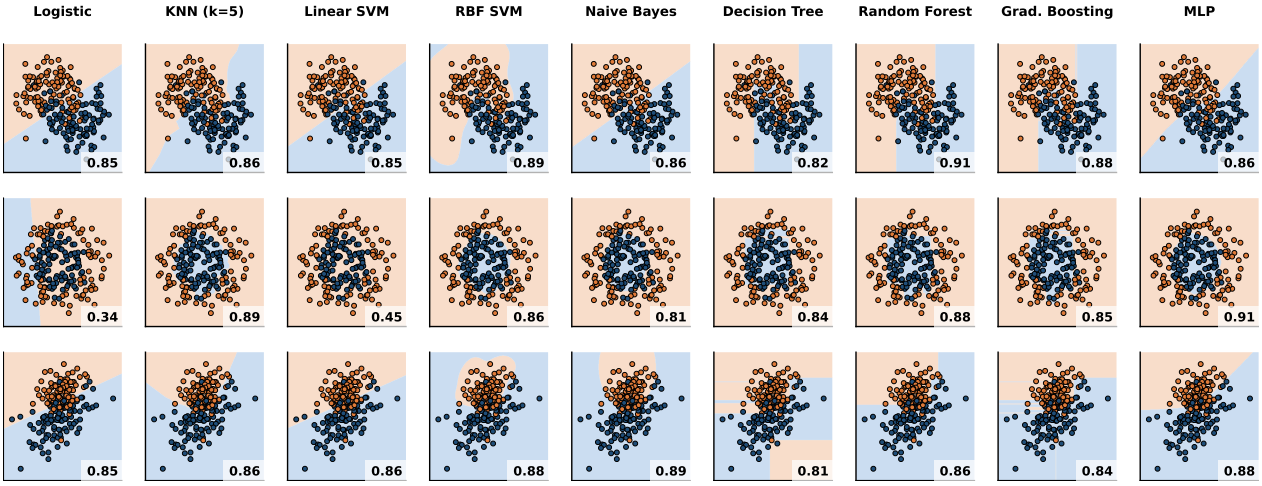


Figure 10.1: Decision boundaries of nine classifiers on three 2-D datasets (moons, circles, linearly separable). Bottom-right number in each panel = test accuracy. Linear models fail on circles; KNN, RBF-SVM, ensembles and MLPs adapt; trees produce axis-aligned staircases.

10.2 Master model table

Model	Task	Best data fit	Scale?	Interpret.	Typical use cases
Linear regression	Reg	Linear relations, $n \gg p$	GD only	High	Baselines, effect sizes, pricing
Ridge / Lasso / EN	Reg	Many / correlated features, $p \geq n$	Yes	High	Genomics, marketing mix, feature selection
Logistic regression	Clf	Linearly separable, sparse text	Yes	High	Credit scoring, churn, CTR, spam
Naive Bayes	Clf	Text counts, categorical, tiny data	No	Medium	Spam, sentiment, ticket routing
KNN	Both	Low-dim, small–medium n	**Yes**	Medium	Recommenders, imputation
LDA / QDA	Clf	Gaussian-like classes	Helpful	Medium	Small multiclass, supervised dim. reduction
SVM (linear)	Both	High-dim sparse	**Yes**	Medium	Text, bioinformatics
SVM (RBF)	Both	Small/medium, complex boundaries	**Yes**	Low	Images (features), handwriting
Decision tree	Both	Mixed tabular, rules needed	No	**Very high**	Policy rules, triage, explanation
Random forest	Both	Tabular, noisy, mixed	No	Medium	Fraud, risk, importance analysis

Model	Task	Best data fit	Scale?	Interpret.	Typical use cases
AdaBoost	Clf	Clean data, weak learners	No	Low	Face detection (historic), simple tabular
Gradient boosting / XGBoost / LightGBM / CatBoost	Both + rank	Tabular – state of the art	No	Low–Med (SHAP)	Competitions, risk, demand, ranking
MLP / deep nets	Both	Large data; images, text, audio	**Yes**	Low	Vision, NLP, speech
K-Means	Clust	Spherical clusters, large n	**Yes**	High	Segmentation, quantisation
Hierarchical	Clust	Small n, need hierarchy	Yes	High	Taxonomies, gene expression
DBSCAN	Clust	Arbitrary shapes, noise	Yes	Medium	Geo/GPS, anomaly detection
GMM	Clust. / density	Elliptical, soft clusters	Yes	Medium	Soft segmentation, density estimation
PCA	Dim. red.	Correlated numeric	**Yes**	Medium	Compression, de-noising, pre-processing
t-SNE / UMAP	Visual.	Non-linear manifolds	Yes	Low	Embedding visualisation
Isolation Forest	Anomaly	High-dim tabular	No	Low	Fraud, intrusion, sensor faults
Apriori / FP-Growth	Assoc.	Transactions	–	High	Market basket, cross-sell

10.3 Training and prediction cost

Model	Training (approx.)	Prediction per sample
OLS (normal equation)	$O(np^2 + p^3)$	$O(p)$
Logistic / linear via GD	$O(np \cdot \text{iterations})$	$O(p)$
Naive Bayes	$O(np)$	$O(pK)$
KNN (brute force)	$O(1)$ (stores data)	$O(np)$
Kernel SVM	between $O(n^2 p)$ and $O(n^3)$	$O(n_{SV}\, p)$
Decision tree	$O(p\, n \log n)$	$O(\text{depth})$
Random forest (B trees)	$O(B\, m\, n \log n)$, parallel	$O(B \cdot \text{depth})$
Gradient boosting (M trees)	$O(M \cdot \text{tree cost})$, sequential	$O(M \cdot \text{depth})$
K-Means	$O(nKp \cdot \text{iterations})$	$O(Kp)$
Agglomerative	$O(n^2 \log n)$ time, $O(n^2)$ memory	–
DBSCAN	$O(n \log n)$ with index, $O(n^2)$ worst	–
PCA (full SVD)	$O(np^2 + p^3)$	$O(pk)$

Model	Training (approx.)	Prediction per sample
t-SNE	$O(n^2)$; Barnes–Hut $O(n \log n)$	– (no transform)

10.4 A practical decision guide

1. **Do you have labels?** No → clustering (K-Means for speed, DBSCAN for odd shapes/noise, GMM for soft clusters), PCA/UMAP for structure, Isolation Forest for anomalies. Yes → continue.
2. **What is the data type?**
   - *Tabular* → start with linear/logistic regression as a baseline, then random forest, then tuned gradient boosting (LightGBM/XGBoost; CatBoost for many categoricals).
   - *Text* → TF-IDF + logistic regression / linear SVM / Multinomial NB; transformers when data and compute allow.
   - *Images, audio* → deep learning (pretrained CNNs/ViTs), not classical models.
   - *Time series* → lag/rolling features + gradient boosting, or dedicated forecasting models; validate with TimeSeriesSplit.
3. **How much data?** Small (<1k rows) → simple, regularised models, SVM, NB, repeated CV. Medium → ensembles. Huge → SGD-based linear models, LightGBM, neural nets.
4. **Must it be explainable?** Yes → linear/logistic, shallow trees, or GBM + SHAP explanations.
5. **Latency / memory constraints?** Avoid KNN and kernel SVMs at prediction time on big data; prefer linear models or small boosted ensembles.
6. **Imbalanced?** Stratify, use PR-AUC/recall/MCC, class weights or SMOTE in-fold, tune the threshold.

10.5 Hyper-parameter quick reference

Model	Most important hyper-parameters	Effect of increasing
Ridge / Lasso	`alpha`	More shrinkage → higher bias, lower variance
Logistic / SVM	`C`	*Less* regularisation → fits training data more closely
RBF SVM	`gamma`	More local, wiggly boundary → overfitting
KNN	`n_neighbors`	Smoother boundary → higher bias
Decision tree	`max_depth`, `min_samples_leaf`	Depth ↑: more variance; leaf size ↑: smoother
Random forest	`n_estimators`, `max_features`	Trees ↑: stabler (no overfit); features ↑: more correlated trees
GBM / XGBoost	`learning_rate`, `n_estimators`, `max_depth`, `subsample`	Rate ↑: faster but riskier; trees ↑: can overfit – use early stopping
K-Means	`n_clusters`	Lower inertia always; validate with silhouette
DBSCAN	`eps`, `min_samples`	eps ↑: clusters merge; min_samples ↑: more noise points
PCA	`n_components`	More variance kept, less compression

CHAPTER 11

Interview Quick Revision

Bias vs variance? Bias = error from wrong assumptions (underfitting); variance = sensitivity to the training sample (overfitting). Expected error = bias^2 + variance + noise.

Why does L1 give sparsity but L2 does not? The L1 constraint region has corners on the axes where loss contours touch first; algebraically, soft-thresholding sets small coefficients exactly to zero while L2 only rescales them.

Why log loss rather than MSE for logistic regression? Log loss is the Bernoulli negative log-likelihood, is convex in the weights, and gives strong gradients for confident mistakes; MSE with a sigmoid is non-convex.

Bagging vs boosting? Bagging trains independent deep models on bootstrap samples in parallel to reduce variance; boosting trains shallow models sequentially on the errors (weights/gradients) of the previous ones to reduce bias.

Why does a random forest subsample features? To de-correlate trees: ensemble variance = $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, and averaging cannot remove the $\rho\sigma^2$ term.

What makes XGBoost different from classical GBM? Second-order (gradient + Hessian) approximation, explicit regularisation $\gamma T + \frac{1}{2}\lambda\|w\|^2$ giving closed-form leaf weights $-G/(H+\lambda)$ and a gain-based pruning rule, plus sparsity handling and system optimisations.

What is the kernel trick? The SVM dual depends on data only through dot products; replacing them with a kernel $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$ fits a linear separator in a high-dimensional feature space without computing ϕ.

Precision vs recall – which matters? Recall when missing a positive is costly (disease, fraud); precision when false alarms are costly (spam filter, manual review queues).

ROC-AUC vs PR-AUC? ROC-AUC measures ranking and is insensitive to imbalance; PR-AUC focuses on the positive class and is more informative when positives are rare.

Why stratified K-fold? It keeps the class ratio in every fold, so each validation fold is representative – vital for imbalanced data.

How do you detect overfitting? Large gap between training and validation scores; validation error increasing while training error decreases; learning curves with a persistent gap.

What is data leakage? Information unavailable at prediction time leaking into training (target leakage, preprocessing fitted on all data, future data in time series). Fix with pipelines and proper splits.

K-Means assumptions? Spherical, similar-size clusters, Euclidean distance, K known; sensitive to scale, outliers and initialisation (use k-means++, several runs).

PCA in one line? Project centred data onto the top eigenvectors of the covariance matrix – the orthogonal directions of maximum variance.

Parametric vs non-parametric? A fixed number of parameters (linear, logistic, NB) vs complexity that grows with the data (KNN, trees, kernel SVM).

How to handle imbalance? Proper metrics, stratification, class weights, in-fold resampling (SMOTE), threshold tuning, ensembles designed for imbalance.

When would you not use accuracy? Imbalanced classes or unequal error costs.

Why scale features? Distance-based, gradient-based, variance-based and regularised models depend on feature magnitudes; trees do not.

Generative vs discriminative? Generative models learn $P(\mathbf{x} \mid y)P(y)$ (NB, LDA, GMM); discriminative models learn $P(y \mid \mathbf{x})$ or the boundary directly (logistic regression, SVM, trees).

What does the OOB score give you? A nearly free validation estimate for bagging/random forests using the 36.8% of rows each tree did not see.

References & Further Reading

1. T. Hastie, R. Tibshirani, J. Friedman. *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.
2. G. James, D. Witten, T. Hastie, R. Tibshirani. *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.
3. C. M. Bishop. *Pattern Recognition and Machine Learning*, Springer, 2006.
4. K. P. Murphy. *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.
5. L. Breiman. Random Forests. *Machine Learning* 45, 2001.
6. J. H. Friedman. Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 2001.
7. Y. Freund, R. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *JCSS*, 1997.
8. T. Chen, C. Guestrin. XGBoost: A Scalable Tree Boosting System. *KDD*, 2016; and the XGBoost documentation, "Introduction to Boosted Trees".
9. G. Ke et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS*, 2017.
10. L. Prokhorenkova et al. CatBoost: unbiased boosting with categorical features. *NeurIPS*, 2018.
11. M. Ester et al. A density-based algorithm for discovering clusters (DBSCAN). *KDD*, 1996.
12. L. van der Maaten, G. Hinton. Visualizing Data using t-SNE. *JMLR*, 2008.
13. F. T. Liu, K. M. Ting, Z.-H. Zhou. Isolation Forest. *ICDM*, 2008.
14. N. Chawla et al. SMOTE: Synthetic Minority Over-sampling Technique. *JAIR*, 2002.
15. D. Kingma, J. Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015.
16. scikit-learn User Guide: Cross-validation, Model evaluation, Ensemble methods, Clustering (scikit-learn.org).